

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Evidenčné číslo: FEI-104376-117218

NÁVRH IMPLEMENTÁCIE VYBRANÝCH ALGORITMOV RIADENIA

Diplomový projekt

Študijný program:	Robotika a kybernetika
Študijný odbor:	Kybernetika
Školiace pracovisko:	Ústav robotiky a kybernetiky
Školiteľ:	Ing. Marián Tárník, PhD.

Bratislava 2026

Bc. Andrii Stoliar



ZADANIE DIPLOMOVEJ PRÁCE

Študent: **Bc. Andrii Stoliar**
ID študenta: 117218
Študijný program: robotika a kybernetika
Študijný odbor: kybernetika
Vedúci práce: Ing. Marián Tárník, PhD.
Vedúci pracoviska: prof. Ing. František Duchoň, PhD.
Miesto vypracovania: Ústav robotiky a kybernetiky

Názov práce: **Návrh implementácie vybraných algoritmov riadenia**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

Práca sa zameriava na samotnú implementáciu algoritmov riadenia pričom sa predpokladá uplatnenie poznatkov z oblasti pokročilých metód automatického riadenia v rámci príslušného študijného programu. Hlavným cieľom práce je zostavenie riadiaceho systému pre vybraný laboratórny dynamický systém vrátane vlastného softvérového návrhu pre dostupný laboratórny hardvér (mikrokontroléry, PLC a procesory pre distribuovaný riadiaci systém). Pre splnenie cieľa práce je tiež potrebné adekvátne uplatňovať postupy z oblasti identifikácie systémov, numerickej simulácie a vyhodnocovania kvality riadenia systémov.

Úlohy:

1. Oboznámte sa s dostupným laboratórnym hardvérom a vypracujte stručnú technickú dokumentáciu vybraných laboratórnych dynamických systémov.
2. Vypracujte krátky prehľad riadiacich algoritmov vybraných pre následnú implementáciu.
3. Navrhnite a realizujte predmetné riadiace systémy a analyzujte kvalitu riadenia.
4. Vyhodnoťte dosiahnuté výsledky.

Termín odovzdania diplomovej práce: 15. 05. 2026
Dátum schválenia zadania diplomovej práce: 06. 11. 2025
Zadanie diplomovej práce schválil: prof. Ing. Jarmila Pavlovičová, PhD. – garantka študijného programu

Podakovanie

Na tomto mieste by som rád podakoval školiteľovi Ing. Mariánovi Tárnikovi, PhD., za odborné vedenie, cenné rady a trpezlivosť počas vypracovania tejto diplomovej práce.

Zároveň ďakujem celému Ústavu robotiky a kybernetiky za odovzdanie cenných vedomostí a skúseností, ktoré mi pomohli rozvíjať sa v oblasti riadenia a automatizácie.

Veľká vďaka patrí aj mojim priateľom, starým rodičom, mame, otcovi, sestre a mojej dáme srdca za ich obrovskú podporu, povzbudenie a pochopenie počas celého štúdia.

Andrii Stoliar
Bratislava, 2026

Anotácia diplomovej práce

Slovenská technická univerzita v Bratislave

Fakulta elektrotechniky a informatiky

Študijný odbor:	kybernetika
Študijný program:	Robotika a kybernetika
Autor:	Bc. Andrii Stoliar
Diplomová práca:	Návrh implementácie vybraných algoritmov riadenia
Školiteľ:	Ing. Marián Tárník, PhD.
Mesiac, rok odovzdania:	Máj, 2026

Kľúčové slová: Lorem ipsum dolor sit amet

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Master Thesis Abstract

Slovak University of Technology in Bratislava
Fakulty of Electrical Engineering and Information Technology

Branch of Study: Cybernetics
Study Programme: Robotics and Cybernetics
Author: Bc. Andrii Stoliar
Thesis Title: Implementation Design of Selected Control Algorithms
Supervisor: Ing. Marián Tárník, PhD.
Month, Year: May, 2026

Keywords: Lorem ipsum dolor sit amet

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Obsah

Úvod	7
1 Vstupný príklad regulácie	8
1.1 PID regulácia v uzavretej spätnej väzbe	8
1.2 Prenosové funkcie ORO a URO a zákon regulácie	8
2 Identifikácia a popis tepelného systému TS05	11
2.1 Popis tepelného systému	11
2.1.1 Popis laboratórneho fyzikálneho modelu	11
2.1.2 Statické vlastnosti tepelného systému	12
2.2 Identifikácia tepelného systému	15
2.2.1 Meranie údajov a ich spracovanie	15
2.2.2 Identifikácia systému pomocou metódy najmenších štvorcov . .	17
2.2.3 Identifikácia systému pomocou optimalizácie prenosovej funkcie	18
2.2.4 Výsledky identifikácie tepelného systému	19
2.2.5 Aproximácia identifikovaných modelov	20
3 PI regulátor pre systém TS05	22
3.1 Štruktúra PI regulátora	22
3.2 Nastavenie parametrov PI regulátora pre systém TS05	23
3.3 Implementácia PI regulátora v MATLABe	25
4 PI regulácia s prepínaním pracovných bodov	29
4.1 Opis a schéma riadenia s použitím prepínača	29
4.1.1 Globálna regulácia po určitý čas	31
4.1.2 Prepínanie na základe detekcie ustálenia	34
5 MRAC regulácia – gradientná metóda	37
5.1 Popis algoritmu	37
5.1.1 Prevod systému do stavového priestoru	39
5.1.2 Pseudokód gradientnej MRAC regulácie (diskrétna realizácia) .	42
5.2 Implementácia MRAC v prostredí MATLAB	45
5.2.1 Simulácia	47
5.2.2 Experiment na tepelnej sústave	49
Zoznam literatúry	50

Úvod

Riadenie procesov je kľúčovou súčasťou moderných priemyselných systémov, ktoré zabezpečujú efektívnu, bezpečnú a spoľahlivú prevádzku širokého spektra technologických zariadení. S rastúcimi nárokmi na kvalitu výroby, energetickú efektívnosť a flexibilitu riadenia sa čoraz väčší dôraz kladie nielen na samotný návrh riadiacich algoritmov, ale aj na ich praktickú implementáciu v reálnych podmienkach.

Hlavným zameraním tejto práce je práve problematika implementácie riadiacich algoritmov na rôznych hardvérových a softvérových platformách. Dôraz nie je kladený výlučne na teoretický návrh regulátorov, ale najmä na spôsob ich realizácie, integrácie do riadiaceho systému a na porovnanie správania algoritmov v simulačnom prostredí a pri riadení reálneho procesu. Takýto prístup umožňuje identifikovať praktické obmedzenia, ktoré sa v čisto teoretickom návrhu často neprejavia.

Štruktúra práce v jej aktuálnej podobe zodpovedá prirodzenému postupu riešenia úloh, ktorý bol zvolený počas samotnej realizácie experimentov. Jednotlivé časti sú preto usporiadané ako postupný proces od identifikácie systému, cez návrh a ladenie riadiacich algoritmov, až po ich overenie v simulácii a pri riadení reálneho zariadenia. Takéto členenie verne odráža praktický spôsob práce a umožňuje sledovať vývoj návrhu riadenia krok za krokom.

V konečnej verzii práce však bude štruktúra upravená tak, aby zodpovedala štandardnému členeniu technicky orientovaných prác. Najskôr bude systematicky predstavený samotný algoritmus riadenia, jeho logika a vzájomné väzby medzi jednotlivými blokmi, v prípade potreby doplnené o pseudokód. Následne bude uvedený podrobný opis riadeného systému, jeho fyzikálne vlastnosti a proces identifikácie. Až v ďalších kapitolách bude spracovaná samotná implementácia riadenia v simulačnom prostredí a na zvolenej hardvérovej platforme.

Z tohto dôvodu bude existujúca, už vypracovaná časť práce dodatočne štrukturálne upravená a reorganizovaná, pričom obsah jednotlivých kapitol zostane zachovaný. Cieľom tejto úpravy je zosúladiť výslednej štruktúry práce s požiadavkami zadania a zároveň zachovanie logickej nadväznosti medzi jednotlivými krokmi návrhu a implementácie riadenia.

1 Vstupný príklad regulácie

Začatie tejto práce bolo zvolené formou opisného úvodného príkladu regulácie, konkrétne štandardného a všeobecne známeho PID regulátora. Predpokladajme, že máme k dispozícii identifikovaný model systému, prípadne uvažujeme priamu implementáciu algoritmu na reálnom systéme. V oboch prípadoch budeme riadený objekt ďalej označovať ako $G_s(s)$.

1.1 PID regulácia v uzavretej spätnej väzbe

Prenosová funkcia PID regulátora v paralelnej forme je definovaná ako

$$C(s) = K_p + \frac{K_i}{s} + K_d s, \quad (1)$$

kde K_p predstavuje proporcionálny zisk, K_i integračný zisk a K_d derivačný zisk regulátora.

Na reguláciu budeme uvažovať štandardnú zápornú spätnú väzbu. Regulačná odchýlka vzniká ako rozdiel medzi požadovanou hodnotou a výstupom systému,

$$e(t) = w(t) - y(t), \quad (2)$$

pričom $w(t)$ je žiadaná hodnota (setpoint) a $y(t)$ je výstup regulovanej sústavy. Výstup regulátora predstavuje akčný zásah $u(t)$, ktorý vstupuje do systému $G_s(s)$.

Pre diskretnú implementáciu s periódou vzorkovania T_s budeme používať zápis

$$e(k) = w(k) - y(k), \quad (3)$$

kde index k označuje diskretný časový krok.

Bloková schéma uzavretého regulačného obvodu má nasledujúci tvar:



Obr. 1: Bloková schéma uzavretého regulačného obvodu so zápornou spätnou väzbou a PID regulátorom

1.2 Prenosové funkcie ORO a URO a zákon regulácie

Predpokladajme, že regulovaný systém má prenosovú funkciu s čitateľom nultého rádu a menovateľom druhého rádu,

$$G_s(s) = \frac{b_0}{s^2 + a_1 s + a_0}. \quad (4)$$

Prenosová funkcia otvoreného regulačného obvodu (ORO) je daná súčinom systému a regulátora,

$$G_{\text{ORO}}(s) = G_s(s) C(s). \quad (5)$$

Po dosadení (1) a (4) dostaneme

$$\begin{aligned} G_{\text{ORO}}(s) &= \frac{b_0}{s^2 + a_1 s + a_0} \left(K_p + \frac{K_i}{s} + K_d s \right) \\ &= \frac{b_0 (K_d s^2 + K_p s + K_i)}{s (s^2 + a_1 s + a_0)}. \end{aligned} \quad (6)$$

Prenosová funkcia uzavretého regulačného obvodu (URO) pri jednotkovej spätnej väzbe je

$$G_{\text{URO}}(s) = \frac{G_{\text{ORO}}(s)}{1 + G_{\text{ORO}}(s)}. \quad (7)$$

Po dosadení (6) platí

$$G_{\text{URO}}(s) = \frac{b_0 (K_d s^2 + K_p s + K_i)}{s (s^2 + a_1 s + a_0) + b_0 (K_d s^2 + K_p s + K_i)}. \quad (8)$$

Póly ORO sú dané koreňmi menovateľa prenosu $G_{\text{ORO}}(s)$, t.j.

$$s (s^2 + a_1 s + a_0) = 0, \quad (9)$$

odkiaľ priamo vyplýva jeden pól v $s_1 = 0$ a zvyšné dva póly sú koreňmi kvadratickej rovnice

$$s^2 + a_1 s + a_0 = 0. \quad (10)$$

Póly URO sú určené koreňmi charakteristického polynómu (menovateľa) uzavretého obvodu,

$$s (s^2 + a_1 s + a_0) + b_0 (K_d s^2 + K_p s + K_i) = 0, \quad (11)$$

pričom po roznásobení dostaneme explicitný tvar tretieho rádu

$$s^3 + (a_1 + b_0 K_d) s^2 + (a_0 + b_0 K_p) s + b_0 K_i = 0. \quad (12)$$

Stabilita je v oboch prípadoch kľúčová, avšak význam sa líši. V prípade ORO sa stabilita posudzuje najmä z hľadiska robustnosti a bezpečnosti návrhu regulátora. V prípade URO je stabilita nevyhnutná podmienka správnej funkcie regulácie. Pre spojitý systém stabilita znamená, že všetky póly $G_{\text{URO}}(s)$ majú zápornú reálnu časť. V opačnom prípade výstup $y(t)$ diverguje, regulačná odchýlka neklesá a systém môže byť v reálnej prevádzke nebezpečný.

Zákon riadenia

V časovej oblasti je PID zákon regulácie možné zapísať ako

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}. \quad (13)$$

Pre praktickú implementáciu v diskretnom čase budeme uvažovať základnú aproximáciu:

$$I(k) = I(k-1) + T_s e(k), \quad (14)$$

$$D(k) = \frac{e(k) - e(k-1)}{T_s}, \quad (15)$$

$$u(k) = K_p e(k) + K_i I(k) + K_d D(k). \quad (16)$$

Z dôvodu fyzikálnych obmedzení akčného zásahu sa v praxi uplatňuje saturácia

$$u(k) = \min(\max(u(k), u_{\min}), u_{\max}). \quad (17)$$

Pseudokód PID regulátora

Nižšie je uvedený ilustračný pseudokód výpočtu PID regulátora v diskretnom čase s vnútornými stavmi integrátora a pamäťou predchádzajúcej chyby.

```
1  % Inputs: w (setpoint), y (output), Ts
2  % States: I (integrator state), e_prev (previous error)
3  % Params: Kp, Ki, Kd, u_min, u_max
4
5  e = w - y;
6
7  % Integrator update
8  I = I + Ts * e;
9
10 % Derivative term
11 D = (e - e_prev) / Ts;
12
13 % PID law
14 u = Kp * e + Ki * I + Kd * D;
15
16 % Saturation
17 u = min(max(u, u_min), u_max);
18
19 % Memory update
20 e_prev = e;
```

Výpis kódu 1: Pseudokód diskretné implementácie PID regulátora (ilustračný príklad)

2 Identifikácia a popis tepelného systému TS05

V tejto kapitole sa budeme zaoberať laboratórnym modelom, ktorý bol poskytnutý pre potreby tejto diplomovej práce – tepelným systémom TS05. Cieľom kapitoly je podrobne opísať túto experimentálnu zostavu, vykonať jej identifikáciu a navrhnúť viaceré riešenia riadenia, a to ako klasické, tak aj pokročilejšie metódy.

Súčasťou práce je tiež implementácia vybraných riadiacich algoritmov na rôznych hardvérových platformách. Táto časť má umožniť porovnanie prístupov k implementácii riadenia v simulačnom prostredí MATLAB/Simulink a na vybranom mikrokontroléri, čím sa vytvorí priestor pre analýzu rozdielov v správaní systému a v samotnej realizácii algoritmov.

2.1 Popis tepelného systému

Táto kapitola sa zaoberá popisom laboratórneho modelu TS05 a jeho základnými statickými vlastnosťami. Model predstavuje fyzický systém určený na výučbu a experimentálne overovanie princípov riadenia v oblasti automatizácie a mechatroniky.

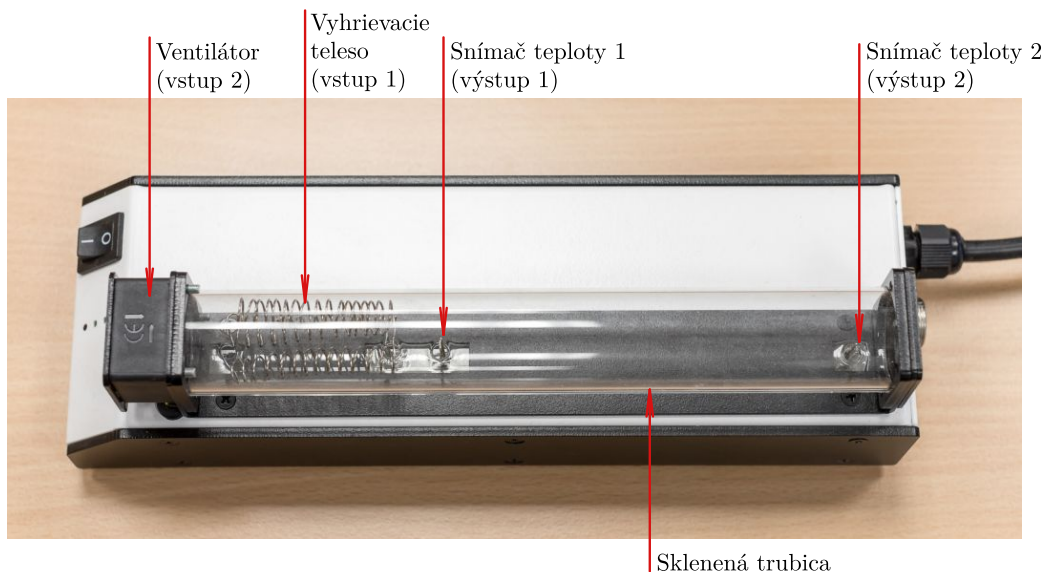
2.1.1 Popis laboratórneho fyzikálneho modelu

Laboratórny model TS05 predstavuje MIMO systém (Multiple Input – Multiple Output) so dvomi vstupmi a dvomi výstupmi, ktorého zapojenie je uvedené v tabuľke 1. Vstupné veličiny tvoria napätia riadiace ventilátor a vykurovaciú špirálu (v rozsahu 0–10 V), zatiaľ čo výstupy sú merané pomocou Senzora 1 a Senzora 2 (tiež v rozsahu 0–10 V).

Tabuľka 1: Vstupy a výstupy systému TS05

Typ signálu	Označenie	Rozsah
Vstup	Ventilátor	0–10 V
Vstup	Výkon špirály	0–10 V
Výstup	Senzor 1 (teplota 1)	0–10 V
Výstup	Senzor 2 (teplota 2)	0–10 V

Vizuálny vzhľad systému je znázornený na obrázku 2.

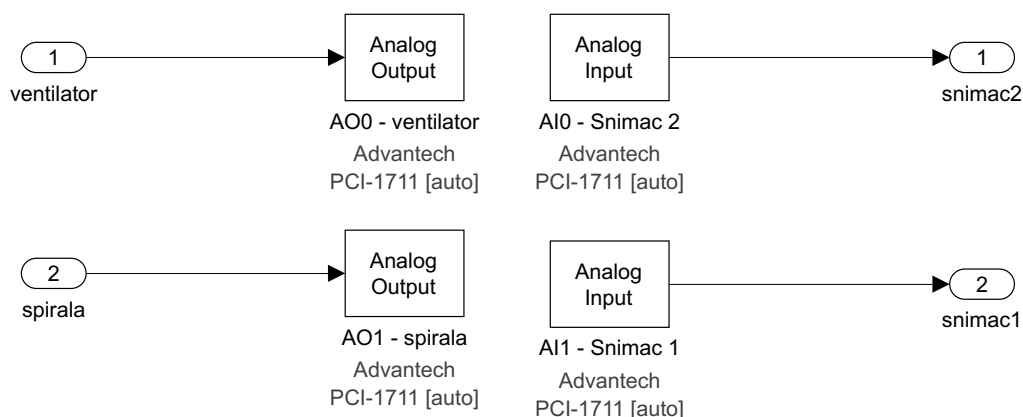


Obr. 2: Laboratórny model TS05 – reálna zostava [2]

Systém komunikuje s počítačom prostredníctvom meracej a komunikačnej karty *Humusoft*, ktorá je určená na prepojenie reálnych systémov so simulačným prostredím

MATLAB/Simulink. Táto karta umožňuje obojsmerný prenos analógových signálov – prijíma merané veličiny zo systému a zároveň vysiela riadiace signály na jeho vstupy.

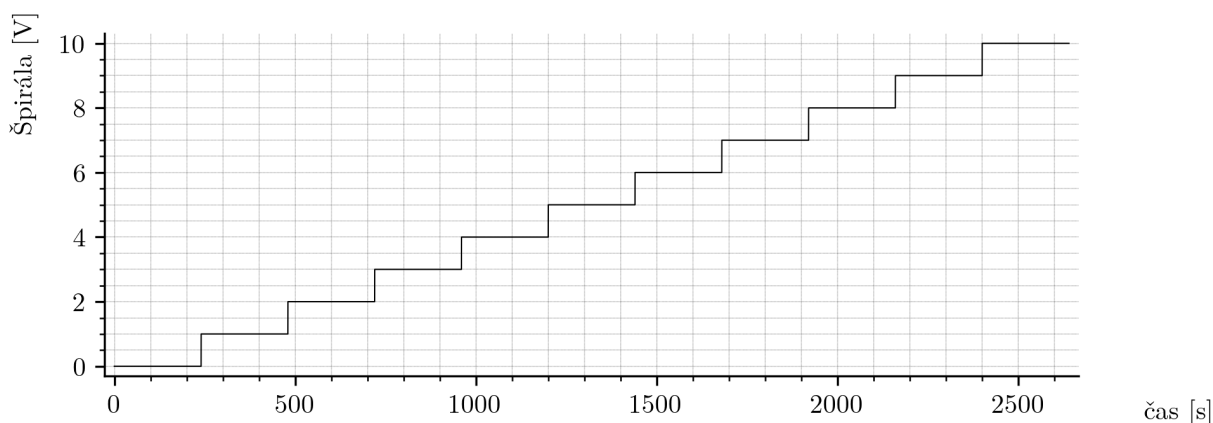
Samotná komunikácia a spracovanie signálov prebieha v prostredí Simulink, v rámci vopred pripravenej schémy (obr. 3), ktorá bola vytvorená ako súčasť projektu KUT – Krátke výučbové materiály. Táto schéma slúži aj ako podklad pre predmet *Modelovanie a riadenie systémov* [2].



Obr. 3: Simulinková schéma pre komunikáciu so systémom TS05

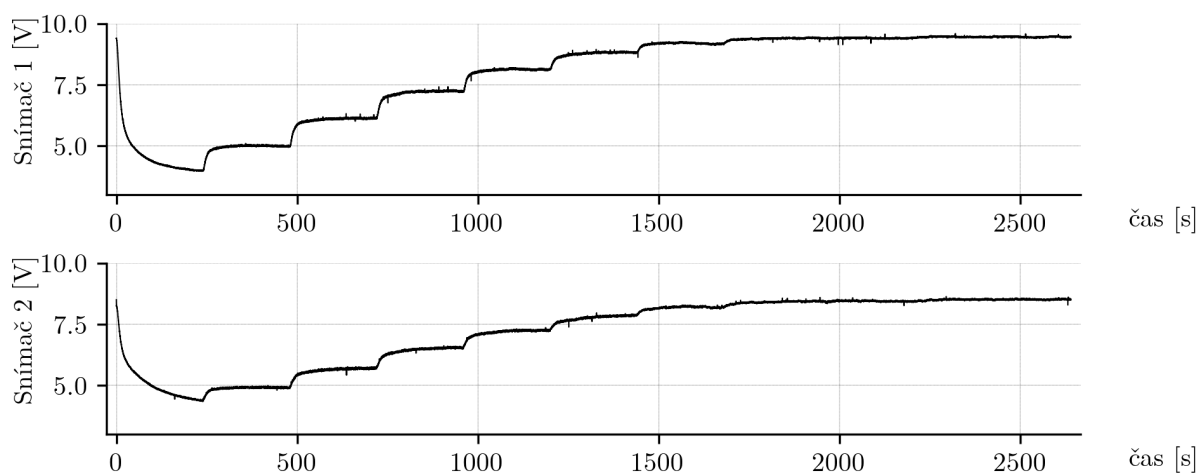
2.1.2 Statické vlastnosti tepelného systému

Na lepšie pochopenie správania systému boli experimentálne namerané jeho statické charakteristiky. Počas meraní boli skúmané tri pracovné body ventilátora – 3 V, 6 V a 9 V – pričom v každom prípade bola hodnota napätia ventilátora udržiavaná konštantná. Napätie na vykurovacej špirále sa menilo lineárne od 0 V do 10 V počas celého merania, keďže špirála má výraznejší vplyv na teplotu ako ventilátor. Priebeh signálu špirály počas experimentov je zobrazený na obrázku 4.

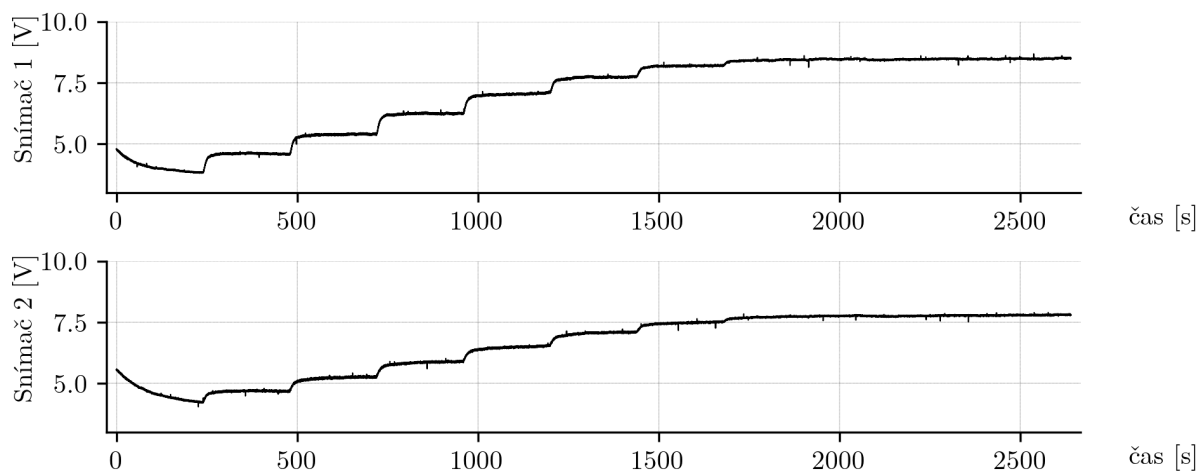


Obr. 4: Priebeh napätia na špirále počas všetkých troch experimentov

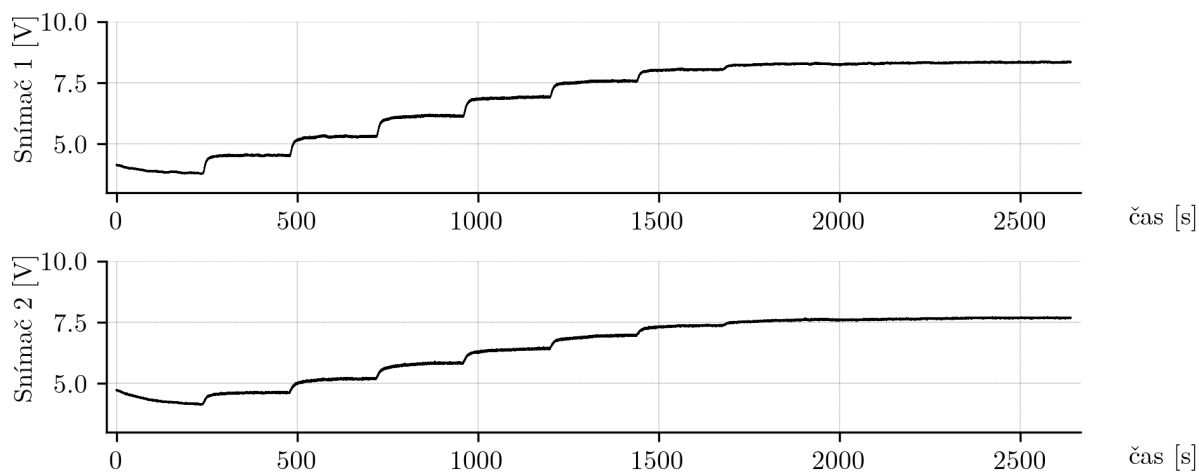
Výsledkom meraní sú šesť grafov, ktoré zobrazujú odozvu systému na zmenu vstupných veličín. Každý graf obsahuje výstupy zo Senzora 1 a Senzora 2 ako funkcie času pre jednotlivé pracovné body ventilátora.



Obr. 5: Odozva systému TS05 pre pracovný bod ventilátora 3 V

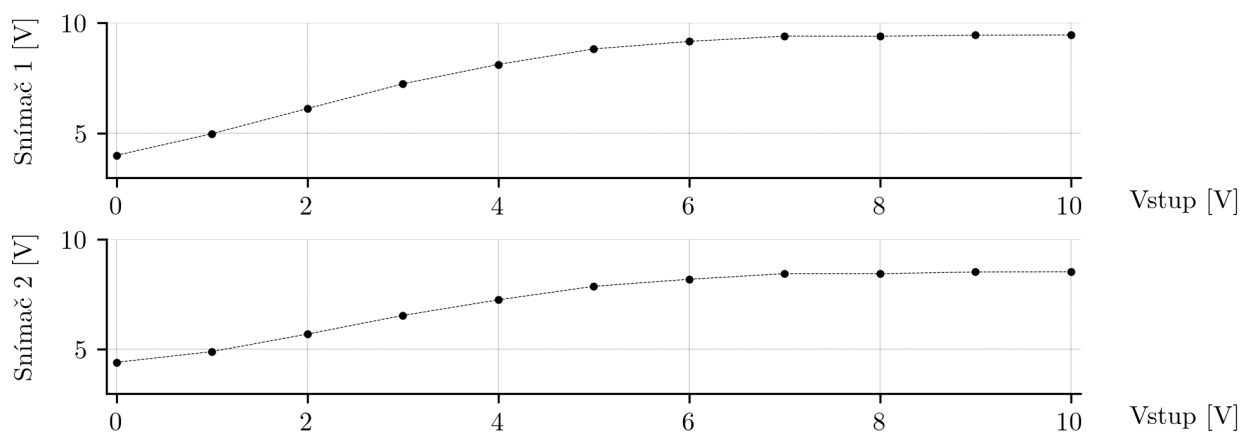


Obr. 6: Odozva systému TS05 pre pracovný bod ventilátora 6 V

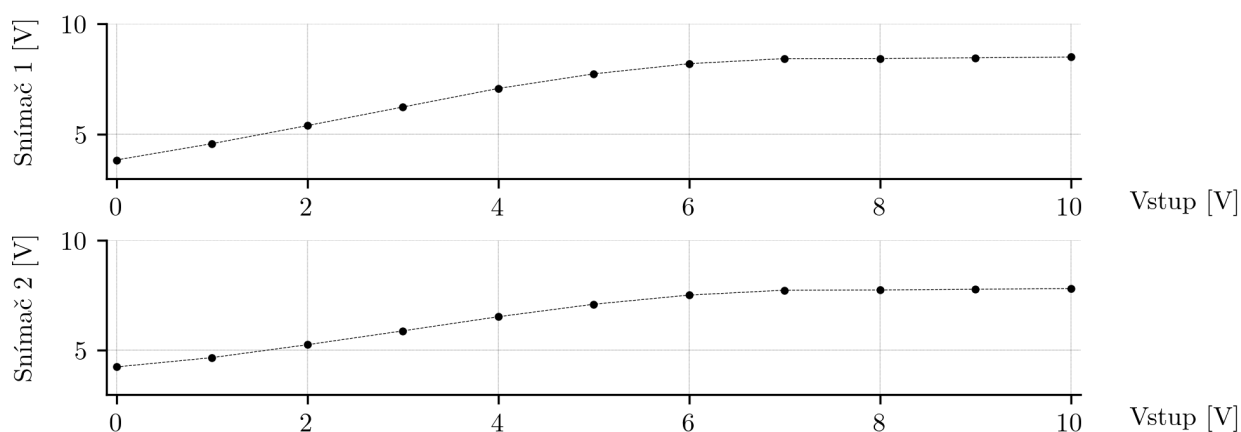


Obr. 7: Odozva systému TS05 pre pracovný bod ventilátora 9 V

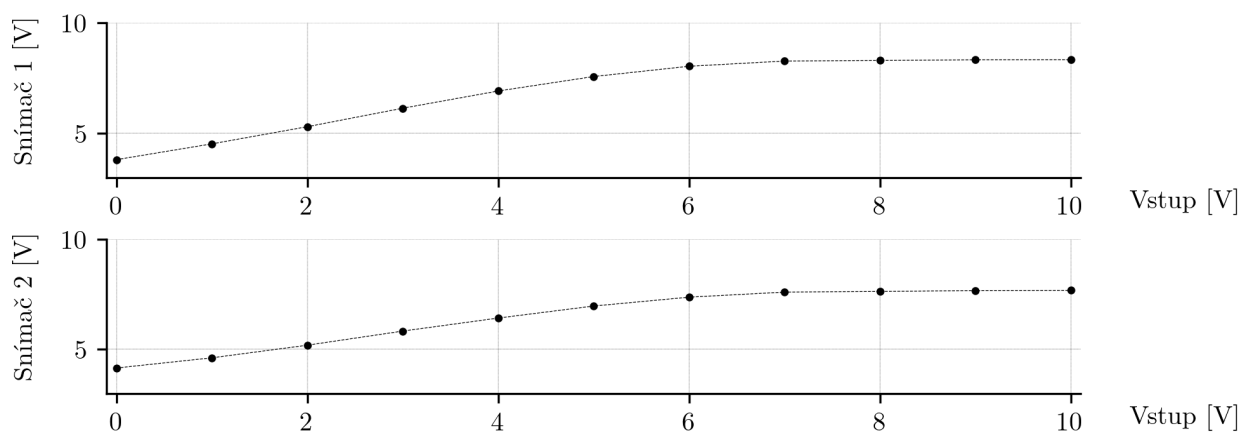
Z tých istých meraní boli zostavené aj prevodové (statické) charakteristiky, ktoré opisujú závislosť výstupných veličín od vstupného napätia špirály pre jednotlivé pracovné body ventilátora.



Obr. 8: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 3 V



Obr. 9: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 6 V



Obr. 10: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 9 V

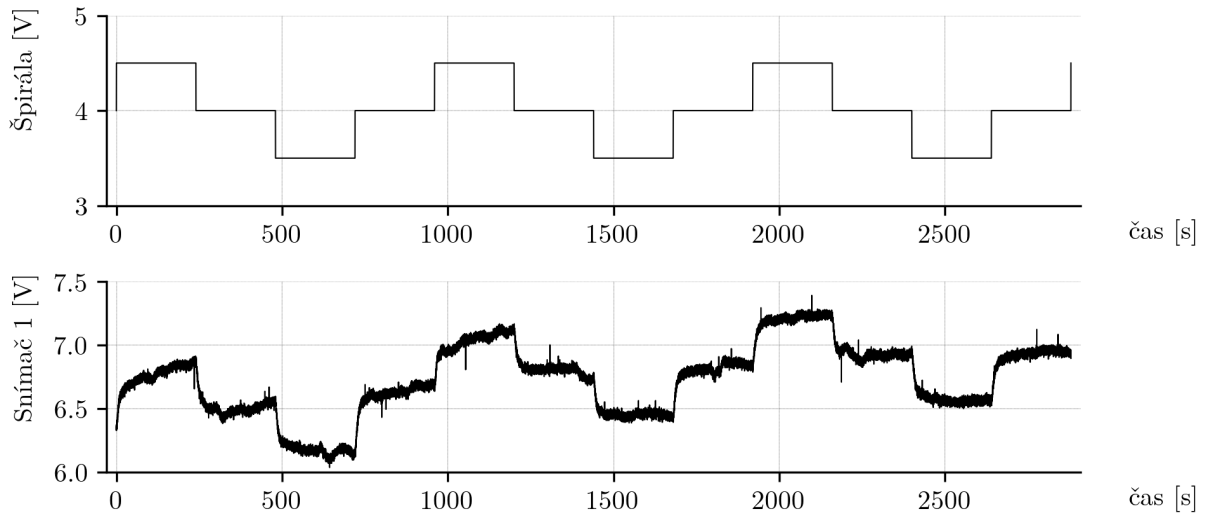
2.2 Identifikácia tepelného systému

V tejto časti sa pokúsime identifikovať tepelný systém TS05 rôznymi metódami a porovnať ich výsledky. Cieľom je určiť, ktoré prístupy poskytujú presnejšie modely a na aké faktory je potrebné dávať pozor počas procesu identifikácie.

2.2.1 Meranie údajov a ich spracovanie

Na účely identifikácie systému akoukoľvek *off-line* metódou je najprv potrebné získať reakciu systému na jednotkové skoky v okolí pracovného bodu. V našom prípade bol pracovný bod zvolený ako vstupné napätia špirály 4 V a ventilátora 6 V.

Na obrázku 11 je zobrazený vstupný signál špirály a odpoveď systému na výstupe zo senzora 1.



Obr. 11: Vstupný signál špirály a výstup senzora 1 pri skokovej zmene

Z grafu možno pozorovať zreteľný lineárny trend, teda postupné zvyšovanie výstupnej teploty počas celého merania. Tento jav je spôsobený pomalým nahrievaním špirály a tiež prirodzenými zmenami teploty v miestnosti, ktoré neboli v rámci experimentu kontrolované. Teplota prostredia sa však počas meraní pohybovala v rozsahu bežnej izbovej teploty, takže systém bol v stabilných podmienkach.

Aby bolo možné získať z nameraných údajov vhodné parametre pre identifikáciu, bolo potrebné odstrániť tento trend aplikovaním odstránenia lineárneho trendu. Metodika výpočtu je založená na lineárnej regresii, kde sa určuje priamka trendu v tvare

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x \quad (18)$$

pričom koeficienty sa počítajú podľa vzťahov [3]:

$$\hat{\beta}_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \quad (19)$$

Takto určený trend je následne odčítaný od pôvodného signálu:

$$y_{\text{detr}} = y - (\hat{\beta}_0 + \hat{\beta}_1 x) \quad (20)$$

Uvedený postup bol implementovaný v prostredí *MATLAB* pomocou vlastnej funkcie:

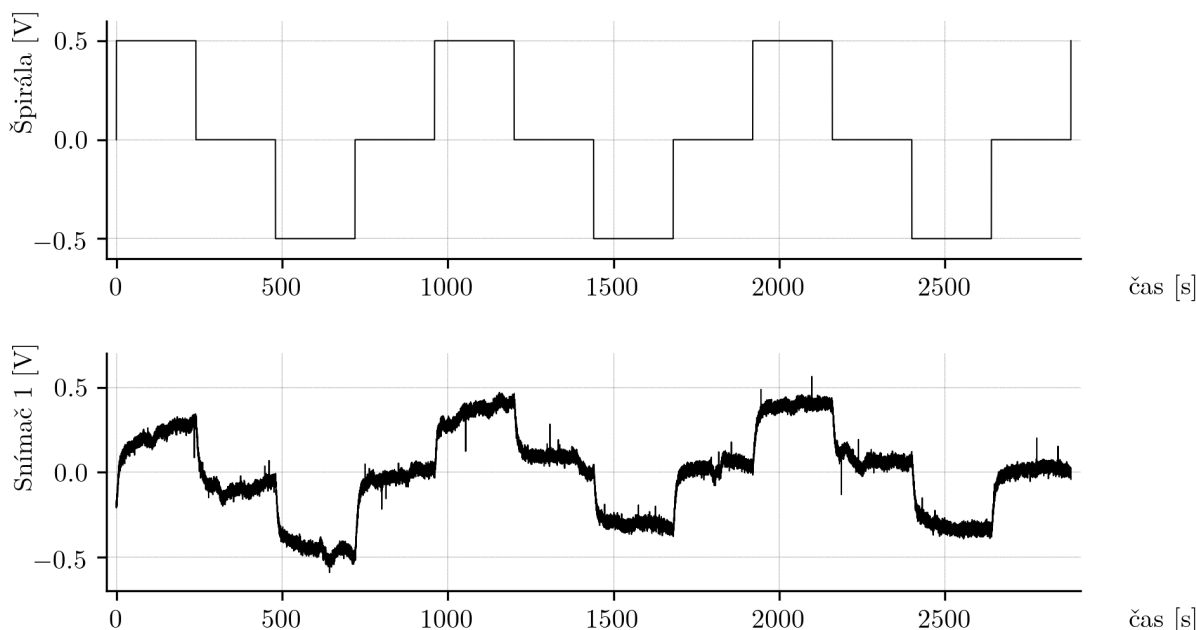
```

1 function y_detr = my_detr(y, x)
2     y = y(:);
3     x = x(:);
4     n = length(y);
5
6     Sx = sum(x);
7     Sy = sum(y);
8     Sxx = sum(x.^2);
9     Sxy = sum(x .* y);
10
11     b1 = (n*Sxy - Sx*Sy) / (n*Sxx - Sx^2);
12     b0 = mean(y) - b1 * mean(x);
13
14     y_detr = y - (b0 + b1*x);
15 end

```

Výpis kódu 2: MATLAB implementácia detrendácie signálu pomocou lineárnej regresie

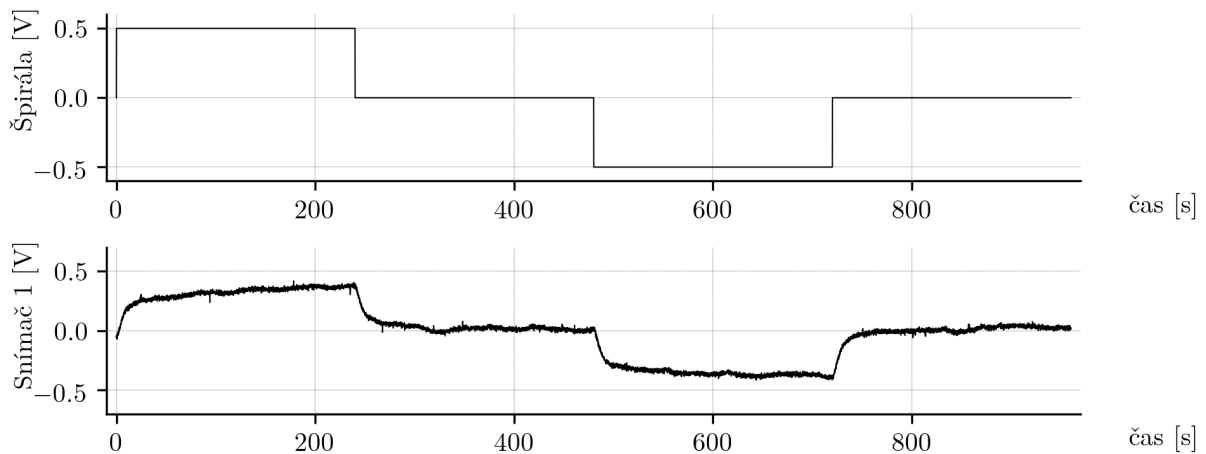
Po odstránení lineárneho trendu bol signál zároveň normalizovaný pre potreby ďalšej identifikácie. Výsledné priebehy normalizovaných a detrendovaných údajov sú zobrazené na obrázku 12.



Obr. 12: Normalizované a detrendované priebehy vstupného a výstupného signálu

Z grafov možno vidieť, že jednotlivé skokové odozvy sa mierne líšia svojím priebehom, čo poukazuje na citlivosť systému TS05 na vonkajšie podmienky a fluktuácie prostredia. Preto boli vykonané tri nezávislé experimenty s rovnakým vstupným signálom, ktorých priebehy sú zobrazené na obrázku 12. Po detrendácii a normalizácii boli tieto dáta spriemerované bod po bode, aby sa eliminoval vplyv náhodných rušení a získala reprezentatívna odozva systému.

Výsledný stredný priebeh, znázornený na obrázku 13, predstavuje typickú reakciu systému TS05 v okolí pracovného bodu 4 V s rozsahom $\pm 0,5$ V a slúži ako podklad pre ďalšiu identifikáciu modelu.



Obr. 13: Priemerná odozva systému TS05 v pracovnom bode 4 V

2.2.2 Identifikácia systému pomocou metódy najmenších štvorcov

V tejto časti bola uskutočnená identifikácia systému TS05 na základe priemernej odozvy získanej z troch experimentálnych meraní popísaných v predchádzajúcej kapitole. Tento priemerný priebeh reprezentuje typickú reakciu systému v okolí pracovného bodu 4 V a slúži ako základ pre odhad parametrov modelu. Cieľom bolo získať lineárny model druhého rádu, ktorý dostatočne presne opisuje dynamické správanie systému v tejto oblasti.

Identifikácia bola realizovaná pomocou metódy najmenších štvorcov [1], ktorá umožňuje odhad parametrov diskrétného modelu systému na základe meraných vstupov a výstupov. Pre odhad parametrov bola vytvorená regresná matica $H(k, :)$ a vektor výstupov Y , pričom výsledný odhad parametrov $\hat{\theta}$ bol určený podľa rovníc:

$$H(k, :) = [-y(k-1), -y(k-2), u(k-1), u(k-2)] \quad (21)$$

$$\hat{\theta} = (H^T H)^{-1} H^T Y \quad (22)$$

Na základe týchto odhadov boli vypočítané koeficienty diskrétného modelu druhého rádu, ktorý následne predstavuje prenosovú funkciu $G_d(z)$. Pre účely simulácie a ďalšieho návrhu riadenia bol tento model prevedený na spojitú formu $G(s)$ pomocou Tustinovej aproximácie.

V prostredí MATLAB bola implementovaná funkcia, ktorá realizuje uvedené výpočty a vytvára identifikovaný model systému. Implementácia tejto funkcie je uvedená nižšie:

```

1 function G = ls_identify_basic(y, u, Ts)
2
3     y = y(:);
4     u = u(:);
5     N = length(y);
6     na = 2; nb = 2;
7
8     H = zeros(N-2, na+nb);
9     for k = 3:N
10         H(k-2,:) = [-y(k-1), -y(k-2), u(k-1), u(k-2)];
11     end
12     Y = y(3:N);
13
14     theta = pinv(H) * Y;
15
16     a1 = theta(1);
17     a2 = theta(2);

```

```

18     b1 = theta(3);
19     b2 = theta(4);
20
21     num = [b1 b2];
22     den = [1 a1 a2];
23     Gd = tf(num, den, Ts);
24
25     G = d2c(Gd, 'tustin');
26
27 end

```

Výpis kódu 3: MATLAB implementácia identifikácie metódou najmenších štvorcov

2.2.3 Identifikácia systému pomocou optimalizácie prenosovej funkcie

V tejto časti bol zvolený menej akademický, skôr prakticky orientovaný prístup k identifikácii systému TS05. Namiesto priameho odhadu parametrov modelu metódou najmenších štvorcov bola použitá optimalizácia parametrov prenosovej funkcie tak, aby sa minimalizovala chyba medzi simulovanou odozvou modelu a reálnou priemernou odozvou systému, získanou z experimentov.

Ako nástroj optimalizácie bola použitá funkcia `fminsearch` z prostredia MATLAB, ktorá implementuje Nelderovu–Meadovu simplexovú metódu. Ide o bezderivačnú lokálnu optimalizačnú metódu, ktorá na základe hodnotenia účelovej funkcie v niekoľkých bodoch parametrov iteratívne posúva simplex (mnohouholník v priestore parametrov) tak, aby sa znižovala hodnota účelovej funkcie. V našom prípade je účelová funkcia definovaná ako koreň zo strednej kvadratickej chyby (RMSE) medzi nameranou odozvou systému a simulovaným výstupom modelu:

$$J(\mathbf{p}) = \sqrt{\frac{1}{N} \sum_{k=1}^N (y(k) - y_{\text{sim}}(k, \mathbf{p}))^2} \quad (23)$$

kde $y(k)$ je nameraný výstup, $y_{\text{sim}}(k, \mathbf{p})$ je výstup simulovaný modelom s parametrami $\mathbf{p}$ a N je počet vzoriek merania.

Optimalizácia prebieha nad parametrami diskrétného modelu druhého rádu, ktoré sú následne prevedené na spojitú prenosovú funkciu pomocou Tustinovej aproximácie. Implementácia použitej funkcie v prostredí MATLAB je uvedená nižšie:

```

1 function G = tf_identify_fmin(y, u, time, Ts)
2
3     y = y(:);
4     u = u(:);
5     time = time(:);
6
7     p0 = [0 0 0 0];
8
9     cost_fun = @(p) rmse_cost_stable(p, u, y, time, Ts);
10
11     opts = optimset('Display', 'off', 'TolX', 1e-8, 'TolFun', 1e-8);
12     p_opt = fminsearch(cost_fun, p0, opts);
13
14     p_opt = 50 * tanh(p_opt);
15     a1 = p_opt(1); a2 = p_opt(2); b1 = p_opt(3); b2 = p_opt(4);
16
17     Gd = tf([b1 b2], [1 a1 a2], Ts);
18     G = d2c(Gd, 'tustin');
19
20 end
21
22 function err = rmse_cost_stable(p, u, y, t, Ts)
23
24     p = 50 * tanh(p);
25     a1 = p(1); a2 = p(2); b1 = p(3); b2 = p(4);
26
27     try
28
29         Gd = tf([b1 b2], [1 a1 a2], Ts);

```

```

30
31     poles = pole(Gd);
32     if any(abs(poles) >= 1)
33         err = 1e6;
34         return;
35     end
36
37     G = d2c(Gd, 'tustin');
38     y_sim = lsim(G, u, t);
39
40     err = sqrt(mean((y - y_sim).^2));
41
42     catch
43         err = 1e6;
44     end
45 end

```

Výpis kódu 4: MATLAB implementácia identifikácie pomocou optimalizácie prenosovej funkcie

Pri tomto prístupe je dôležité upozorniť na jeden podstatný problém. Ak nie sú parametre prenosovej funkcie počas optimalizácie vhodne obmedzené, algoritmus môže nájsť také hodnoty parametrov, ktoré síce veľmi dobre aproximujú konkrétnu meranú odozvu, ale vedú k nerealistickému alebo nestabilnému správaniu modelu pri iných vstupoch. V predchádzajúcich pokusoch viedlo práve toto k prenosovým funkciám s „nezdravými“ parametrami, ktoré boli použiteľné iba pre konkrétny fitovaný experiment.

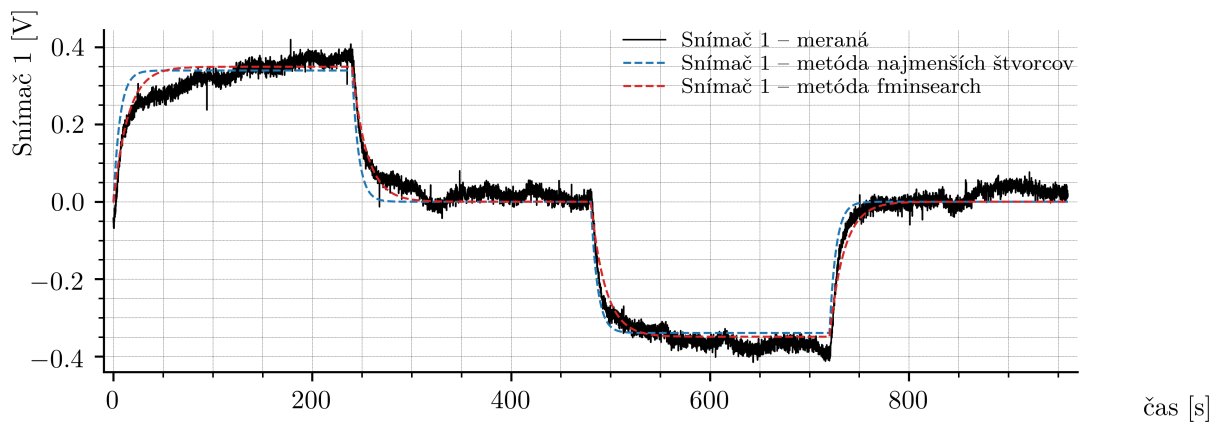
Z tohto dôvodu sú v uvedenej implementácii parametre počas optimalizácie mapované pomocou transformácie

$$p_i^* = 50 \tanh(p_i) \quad (24)$$

čím sú efektívne udržiavané v konečnom rozsahu a znižuje sa riziko vzniku numericky extrémnych alebo fyzikálne nezmyselných modelov. Okrem toho bola do optimalizácie pridaná aj dodatočná penalizácia, ktorá výrazne zvyšuje hodnotu účelovej funkcie v prípade, že identifikovaná prenosová funkcia vykazuje známky nestability. Týmto spôsobom algoritmus uprednostňuje iba stabilné riešenia a zabraňuje vzniku modelov so „zdravotne“ nevhodnými pólmi.

2.2.4 Výsledky identifikácie tepelného systému

Po identifikácii systému v pracovnom bode pomocou dvoch rôznych metód (metódy najmenších štvorcov a optimalizácie prenosovej funkcie) boli získané nasledujúce výsledky.



Obr. 14: Porovnanie skokovej odozvy reálneho systému TS05 a identifikovaných modelov

Porovnanie priemernej skokovej odozvy reálneho systému a odoziev oboch identifikovaných modelov je zobrazené na obrázku 14.

Identifikovaný model získaný metódou najmenších štvorcov má tvar

$$G_{s,LS}(s) = \frac{0.00171 s^2 - 0.3379 s + 6.074}{s^2 + 59.01 s + 8.95} \quad (25)$$

zatým čo model identifikovaný pomocou optimalizácie parametrov prenosovej funkcie metódou *fminsearch* je

$$G_{s,fmin}(s) = \frac{0.0007181 s^2 - 0.05942 s + 0.9012}{s^2 + 18.94 s + 1.291} \quad (26)$$

Kvalita zhody medzi modelmi a experimentálnymi dátami bola hodnotená pomocou ukazovateľa RMSE:

$$RMSE_{LS} = 0.036792, \quad (27)$$

$$RMSE_{fmin} = 0.026996 \quad (28)$$

Ako je vidieť, oba modely poskytujú veľmi uspokojivú aproximáciu dynamiky systému v okolí zvoleného pracovného bodu, pričom model získaný metódou *fminsearch* dosahuje o niečo menšiu hodnotu RMSE. Treba však zdôrazniť, že oba identifikované modely sú len druhého rádu, čo predstavuje rozumný kompromis medzi jednoduchosťou a presnosťou pre následný návrh regulácie.

2.2.5 Aproximácia identifikovaných modelov

Na základe získaných prenosových funkcií možno pozorovať, že ich čitatele obsahujú veľmi malé koeficienty pri prvom a druhom rade. Prítomnosť týchto členov je s veľkou pravdepodobnosťou len výsledkom numerických „artefaktov“ vzniknutých pri prevode z diskkrétnej prenosovej funkcie na spojitú formu. Zahrnutie týchto vyšších rádov v čitateli by výrazne komplikovalo niektoré analytické metódy návrhu regulátorov, napríklad metódu umiestnenia pólov.

Pre účely ďalšej práce sme sa preto rozhodli tieto prenosové funkcie aproximovať jednoduchšou formou. Najzrejmější prístup spočíva v tom, že malé koeficienty pri s a s^2 v čitateli zanedbáme a overíme vplyv tejto úpravy pomocou simulácie.

Výsledkom tejto aproximácie sú nasledujúce prenosové funkcie:

$$G_{s,LS,approx}(s) = \frac{6.074}{s^2 + 59.01 s + 8.95} \quad (29)$$

póly:

$$s_1 = -58.8550, \quad s_2 = -0.1521 \quad (30)$$

a

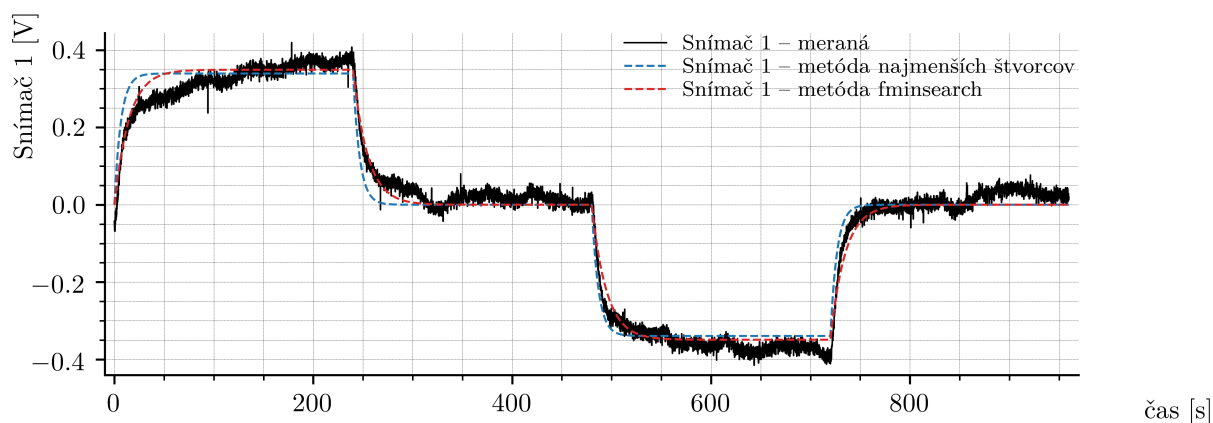
$$G_{s,fmin,approx}(s) = \frac{0.9012}{s^2 + 18.94 s + 1.291} \quad (31)$$

póly:

$$s_1 = -18.8736, \quad s_2 = -0.0684 \quad (32)$$

Z uvedených hodnôt pólov je zrejmé, že obidva modely majú póly s negatívnymi reálnymi časťami, teda ide o stabilné systémy.

Následne bola uskutočnená simulácia odozvy systému analogická tej, ktorá bola uvedená na obrázku 14. Porovnanie reálnej odozvy systému TS05 s oboma aproximovanými modelmi je zobrazené na obrázku 15.



Obr. 15: Porovnanie skokovej odozvy reálneho systému TS05 a aproximovaných modelov

Výsledné hodnoty strednej kvadratickej chyby (RMSE) medzi reálnym systémom a simulovanými odozvami boli:

$$\text{RMSE}_{\text{LS,approx}} = 0.036951, \quad (33)$$

$$\text{RMSE}_{\text{fmin,approx}} = 0.026998 \quad (34)$$

Z výsledkov, ako aj z priebehov simulácií, je zrejmé, že rozdiel medzi pôvodnými a aproximovanými modelmi je zanedbateľný. Hodnota RMSE sa zhoršila len minimálne, čo je z pohľadu ďalšieho použitia modelov úplne prijateľné, najmä vzhľadom na zjednodušenie analytického spracovania, ktoré táto aproximácia prináša.

3 PI regulátor pre systém TS05

V tejto kapitole je riešený návrh a implementácia PI regulátora ako základného nástroja pre reguláciu tepelnej sústavy. Najskôr je stručne zhrnutá teoretická štruktúra PI regulátora a jeho zapojenie v uzavretom regulačnom obvode, následne je tento prístup aplikovaný na systém TS05 vrátane ladenia parametrov a realizácie v reálnom čase.

3.1 Štruktúra PI regulátora

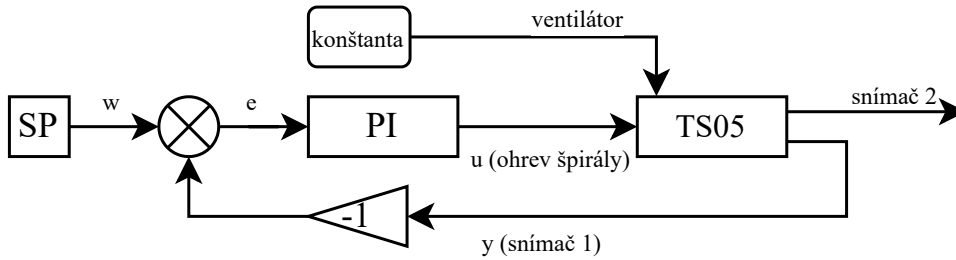
PI regulátor predstavuje zjednodušenú formu PID regulátora, v ktorej je derivačná zložka vynechaná. Pre systémy s pomalou dynamikou, ako sú tepelné sústavy, derivačný člen spravidla neprináša zlepšenie regulácie a naopak môže zosilňovať merací šum.

Paralelná forma PI regulátora je definovaná prenosovou funkciou

$$C(s) = K_p + \frac{K_i}{s}, \quad (35)$$

kde K_p je proporcionálny zisk a K_i integračný zisk regulátora.

Regulátor je zapojený v štandardnej uzavretej spätnej väzbe so zápornou spätnou väzbou. Bloková schéma uzavretého regulačného obvodu je znázornená na obr. 16.



Obr. 16: Bloková schéma uzavretého regulačného obvodu s PI regulátorom

Prenosy regulačného obvodu

Nech $G_s(s)$ označuje prenos regulovaného procesu a $C(s)$ prenos regulátora. Prenos otvoreného regulačného obvodu je potom daný vzťahom

$$G_{ORO}(s) = C(s) G_s(s), \quad (36)$$

pričom prenos uzavretého regulačného obvodu má tvar

$$G_{URO}(s) = \frac{C(s) G_s(s)}{1 + C(s) G_s(s)}. \quad (37)$$

Stabilita uzavretého regulačného obvodu je daná koreňmi charakteristickej rovnice

$$1 + C(s) G_s(s) = 0. \quad (38)$$

Pre spojitý systém je podmienkou stability, aby všetky póly uzavretého obvodu mali zápornú reálnu časť.

Zákon riadenia

V časovej oblasti možno PI regulátor zapísať v tvare

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau, \quad (39)$$

kde $e(t) = w(t) - y(t)$ predstavuje regulačnú odchýlku, $w(t)$ žiadanú hodnotu a $y(t)$ výstup systému.

Pre diskretnú realizáciu s periódou vzorkovania T_s je integračný člen aproximovaný rekurentným vzťahom

$$e(k) = w(k) - y(k), \quad (40)$$

$$I(k) = I(k-1) + T_s e(k), \quad (41)$$

$$u(k) = K_p e(k) + K_i I(k). \quad (42)$$

Z dôvodu fyzikálnych obmedzení akčného zásahu je v praxi uplatnená saturácia riadiaceho signálu.

Pseudokód PI regulátora

```

1 % Inputs: w (setpoint), y (output), Ts
2 % States: I (integrator state)
3 % Params: Kp, Ki, u_min, u_max
4
5 e = w - y;
6
7 % Integrator update
8 I = I + Ts * e;
9
10 % PID law
11 u = Kp * e + Ki * I;
12
13 % Saturation
14 u = min(max(u, u_min), u_max);

```

3.2 Nastavenie parametrov PI regulátora pre systém TS05

Pre získanie parametrov regulátora bola využitá funkcia `pidentune` v prostredí MATLAB, ktorá vykonáva automatizované ladenie regulátora. Tento postup bol zvolený preto, že viaceré analytické metódy ladenia nie je možné na systém TS05 aplikovať.

Charakteristický polynóm uzavretého regulačného obvodu pre PI regulátor má po dosadení tvar

$$s^3 + \frac{b_1}{b_2}s^2 + \left(\frac{b_0}{b_2} + \frac{a_0 K_p}{b_2}\right)s + \frac{a_0 K_i}{b_2} = 0 \quad (43)$$

Ak by sme chceli použiť metódu umiestnenia pólov pomocou rozšíreného želaného polynómu tretieho rádu, jeho všeobecný tvar je

$$s^3 + (p + 2k\omega_0)s^2 + (2k\omega_0 p + \omega_0^2)s + \omega_0^2 p = 0 \quad (44)$$

Porovnaním koeficientov polynómov (43) a (44) dostaneme sústavu rovníc:

$$1 = 1 \quad (45)$$

$$p + 2k\omega_0 = \frac{b_1}{b_2} \quad (46)$$

$$2k\omega_0 p + \omega_0^2 = \frac{b_0}{b_2} + \frac{a_0 K_p}{b_2} \quad (47)$$

$$\omega_0^2 p = \frac{a_0 K_i}{b_2} \quad (48)$$

Táto sústava nemá realizovateľné riešenie pre parametre K_p a K_i . Dôvodom je, že členy závislé od K_p a K_i sa vyskytujú iba v dvoch rovniciach, zatiaľ čo parametre želaného dynamického správania k , ω_0 , p ovplyvňujú tri rovnice. Sústava je preto pre fyzikálne konzistentné riešenie preurčená a neexistuje v nej žiadna kombinácia parametrov, ktorá by spĺňala všetky rovnice súčasne.

Rovnako nie je možné použiť metódu Ziegler–Nichols, keďže námi identifikovaný model (29) nemá kritický bod, ktorý je pre túto metódu potrebný.

Keďže návrh PI regulátora nie je hlavnou témou práce, bol zvolený numerický nástroj `pdtune`. Jeho princíp spočíva v tom, že pre zvolenú požadovanú šírku pásma uzavretého systému vypočíta optimálne hodnoty parametrov PI regulátora tak, aby bola zabezpečená dobrá dynamická odozva a primeraná robustnosť.

Najskôr bol systém prevedený do diskkrétnej podoby:

```
1 Gs = Gs_fmin;
2 Ts = 0.1;
3
4 Gz = c2d(Gs, Ts, 'tustin');
5 [Ad,Bd,Cd,Dd] = ssdata(Gz);
```

Následne bolo vykonané ladenie PI regulátora:

```
1 wc = 0.5;
2 Cz = pdtune(Gz, 'pi', wc);
3
4 [numC,denC] = tfdata(Cz,'v');
5 [Ac,Bc,Cc,Dc] = tf2ss(numC,denC);
```

Získané parametre regulátora:

$$K_p = 3.58, \quad K_i = 1.84$$

Prenosová funkcia otvoreného regulačného obvodu je

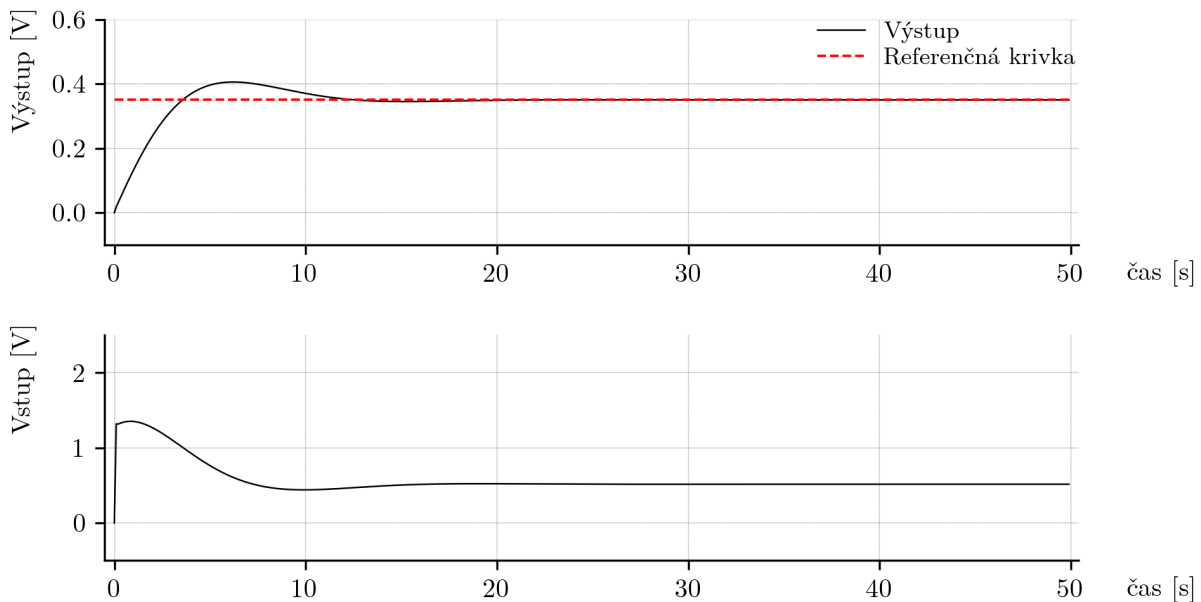
$$G_{oro}(s) = \frac{21.21s + 11.15}{s^3 + 59.01s^2 + 8.95s} \quad (49)$$

Póly otvoreného systému sú:

$$s_1 = 0, \quad s_2 = -58.855, \quad s_3 = -0.1521 \quad (50)$$

Znakom stability je prítomnosť pólov s negatívnou reálnou časťou, čo je splnené.

Simulácia bola vykonaná s obmedzením akčného zásahu na interval $[-4, 6]$, čo zodpovedá pracovnému bodu 4 V. Výsledok simulácie je uvedený na obrázku 17.



Obr. 17: Simulačný výsledok uzavretého PI riadenia systému TS05

Z výsledkov boli namerané tieto hodnoty:

$$\text{Čas ustálenia} = 11.5 \text{ s} \quad (51)$$

$$\text{Prekmit} = 15.9\% \quad (52)$$

$$\text{MSE} = 0.04369 \quad (53)$$

Dosiahnutý výsledok možno považovať za vhodný pre riadenie tepelného systému. Cieľom je dosiahnuť čo najrýchlejšiu odozvu. Prekmit približne 16 percent je akceptovateľný, keďže riadený skok je malý a tepelná sústava je pomalá, čo prirodzene limituje tvar dynamickej odozvy.

3.3 Implementácia PI regulátora v MATLABe

Ako je uvedené v zadaní, cieľom práce je implementácia riadiacich algoritmov na rôznych hardvérových a softvérových platformách. Preto bol PI regulátor implementovaný tak, aby mohol pracovať v reálnom čase bez použitia hotových Simulink blokov určených na riadenie. Celá logika riadenia bola zapísaná priamo do bloku MATLAB Function, ktorý sa vykonáva raz za každý riadiaci cyklus s periódou vzorkovania 0.1 sekundy.

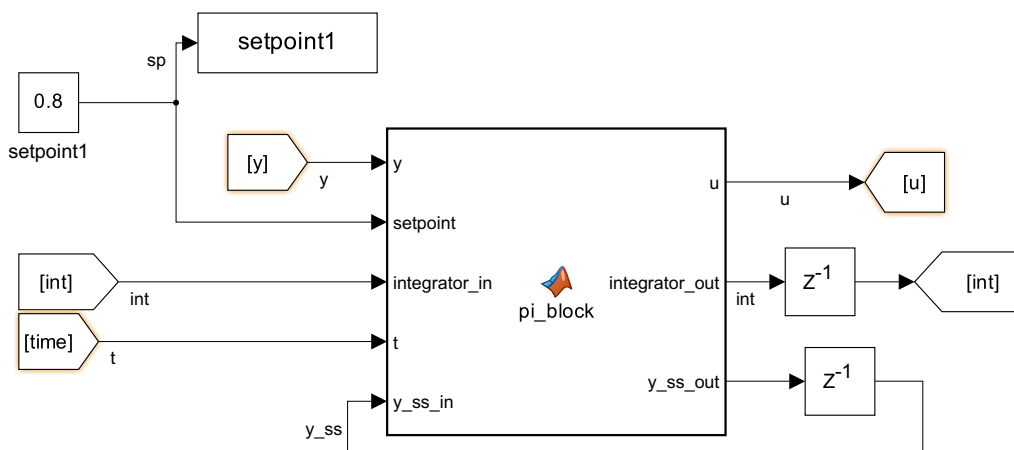
Implementácia regulátora je nasledovná:

```
1 function [u, integrator_out, y_ss_out] = ...
2     pi_block(y, setpoint, integrator_in, t, y_ss_in)
3
4 % PI controller
5
6 pb_u = 4;
7 Ts = 0.1;
8 u_min = - pb_u;
9 u_max = 10 - pb_u;
10
11 Kp = 3.58;
12 Ki = 1.84;
13
14 % waiting phase for steady-state detection
15 if t < 10*60
16     u = 4;
17     y_ss_out = y;
18     integrator_out = 0;
19
20 else
21     y_ss_out = y_ss_in;
22     pb_y = y_ss_in;
23
24     % output normalization
25     y = y - pb_y;
26
27     % control error
28     e = setpoint - y;
29
30     % integrator
31     integrator_out = integrator_in + e * Ts;
32
33     % PI control law
34     u = Kp * e + Ki * integrator_out;
35     u = min(max(u, u_min), u_max);
36
37     % input normalization
38     u = u + pb_u;
39
40 end
41 end
```

Tento blok však na rozdiel od štandardných Simulinkových riadiacich blokov neobsahuje vlastnú vnútornú pamäť, preto je potrebné stavové premenné prenášať medzi jednotlivými volaniami bloku explicitne.

Z tohto dôvodu sú hodnoty integrátora a odhad ustáleného stavu odosielané na výstup PI bloku a v nasledujúcom kroku sú privádzané späť na vstup. Týmto spôsobom sa zabezpečí, že regulátor si zachováva svoje vnútorné stavy z jedného riadiaceho kroku do druhého.

Logika zapojenia bloku MATLAB Function je zobrazená na obrázku 18, ktorý ukazuje vstupy, výstupy a spätnú väzbu stavových premenných potrebných pre správnu činnosť regulátora.



Obr. 18: Zapojenie PI regulátora v bloku MATLAB Function so spätnou väzbou stavov

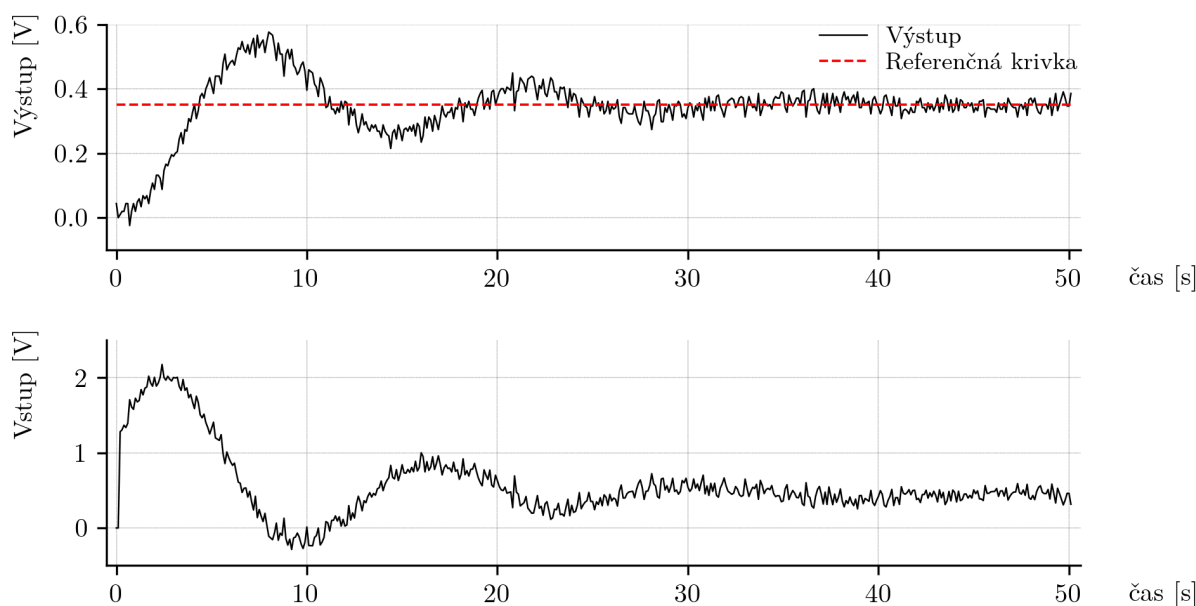
Pred samotným začiatkom regulácie bol zavedený časový úsek, počas ktorého regulátor nevykonáva žiadny zásah. Počas prvých desiatich minút sa na výstupe drží hodnota pracovného bodu a zároveň sa sleduje teplota na výstupe systému. Cieľom je získať reálnu hodnotu ustáleného stavu priamo z aktuálnych podmienok experimentu.

Použitie hodnôt zosadených zo statických charakteristík (obr. 9) nie je vhodné. Tepelný systém je citlivý na teplotu v miestnosti a experimenty nebolo možné realizovať v rovnakých podmienkach. Okrem toho surové merania (obr. 11) ukázali, že teplota narastá veľmi pomaly aj počas dlhého experimentu, čo znamená, že systém nemá pevnú, opakovateľnú hodnotu ustáleného stavu.

Z toho dôvodu je ustálený stav zisťovaný automaticky priamo regulátorom ako posledná nameraná hodnota y pred spustením PI riadenia. Tento prístup funguje bez ohľadu na to, či je systém iba málo zahriaty alebo už dosiahol vysokú prevádzkovú teplotu.

Experiment 1: Regulácia okolo jedného pracovného bodu

Na základe implementácie uvedenej vyššie bol následne vykonaný experiment v simulačnom prostredí s cieľom overiť kvalitu regulácie. Priebeh regulácie je zobrazovaný na obrázku 19.



Obr. 19: Simulačný experiment s PI regulátorom pri skokovej zmene žiadanej hodnoty

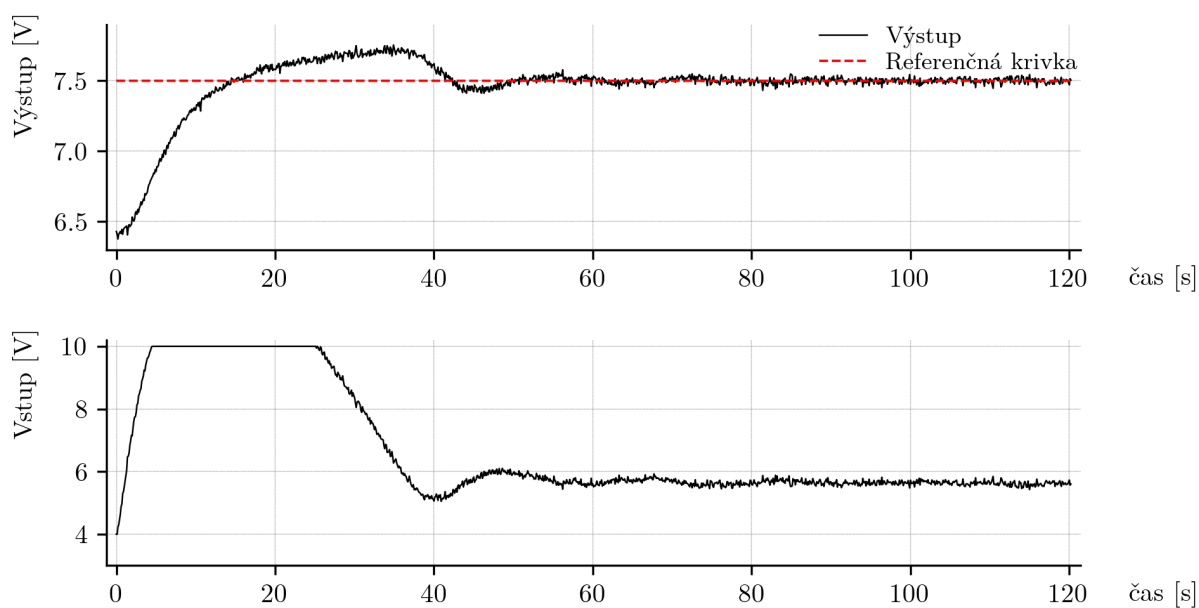
Kvalita regulácie bola hodnotená podľa troch kritérií:

- prekmit: žiadaná hodnota bola 0.35 V, maximálna hodnota výstupu dosiahla približne 0.52 V, čo zodpovedá približne 49.0 percentnému prekmitu,
- čas ustálenia: výstup sa ustálil približne po 28 sekundách,
- trvalá regulačná odchýlka: prakticky nulová.

Pri porovnaní so simuláciou uvedenou v predchádzajúcich kapitolách možno vidieť, že prekmit je výrazne vyšší. Dôvodom je skutočnosť, že identifikované modely nedokázali zachytiť celú dynamiku systému TS05 a viacero pomalých javov, ktoré sa prejavujú až pri reálnej prevádzke. Napriek tomu je regulačný čas veľmi priaznivý vzhľadom na to, že ide o pomalý tepelný systém. Rýchla regulácia bola prioritou, preto možno výsledok považovať za uspokojivý pre riadenie okolo jedného pracovného bodu.

Experiment 2: Reakcia na väčší skok

Nasledoval druhý experiment, ktorého cieľom bolo overiť správanie regulátora pri väčšej skokovej zmene žiadanej hodnoty, ktorá má simulovať prechod medzi rôznymi pracovnými bodmi systému. Systém bol najprv ustálený v pracovnom bode s výstupnou hodnotou približne 6.4 V a následne bola žiadaná hodnota skokovo zmenená na 7.5 V. Výsledok je zobrazený na obrázku 20.



Obr. 20: Experiment PI regulátora pri skoku z 6.4 V na 7.5 V

Z grafu je zrejmé, že počas regulácie došlo k výraznému presýteniu akčného zásahu. Ovládací signál dosiahol svoj limit a počas určitého času na ňom zotrval. Aj keď bola požadovaná hodnota dosiahnutá, takéto správanie je z pohľadu bezpečnosti a kvality regulácie nevyhovujúce.

Z tohto dôvodu možno konštatovať, že navrhnutý PI regulátor je vhodný na riadenie v blízkosti jedného pracovného bodu, avšak nie je vhodný na prepínanie medzi vzdialenejšími pracovnými bodmi. V ďalšej časti práce bude preto implementovaný postup umožňujúci bezpečné a stabilné prepínanie medzi pracovnými bodmi a následne budú jednotlivé regulátory skombinované tak, aby systém disponoval širším využiteľným rozsahom riadenia.

4 PI regulácia s prepínaním pracovných bodov

V tejto časti sa budeme zaoberať návrhom a testovaním prepínača pracovných bodov. Prepínač bude predstavovať pomalý regulátor, ktorého úlohou bude zabezpečiť robustné prechody medzi rôznymi pracovnými bodmi systému TS05. Cieľom je vytvoriť taký spôsob riadenia, ktorý umožní spoľahlivý presun z ľubovoľného pracovného bodu do ľubovoľného iného pracovného bodu bez neželaných dynamických javov, ako je presýtenie akčného zásahu alebo príliš veľké prechodové správanie.

Okrem samotného prepínača bola v tejto časti rozšírená aj pracovná oblasť regulácie o dva ďalšie pracovné body systému – pre napätia špirály 2 V a 6 V. Pre každý z týchto pracovných bodov bol postup opakovaný presne podľa postupu uvedeného v sekcii 2.2, teda boli:

- vykonané merania reakcií systému na jednotkové skoky ± 0.5 V okolo pracovného bodu,
- z nameraných údajov bola identifikovaná prenosová funkcia v danom pracovnom bode,
- pre každý pracovný bod bol navrhnutý PI regulátor s vlastnými parametrami,
- boli určené hodnoty y_{pb} a rozsah výstupu pre ± 0.5 V zmenu vstupu.

Prehľad všetkých troch pracovných bodov je uvedený v tabuľke 2, kde sú uvedené identifikované prenosové funkcie, hodnoty pracovného bodu y_{pb} a parametre jednotlivých PI regulátorov.

Tabuľka 2: Parametre pracovných bodov, identifikované prenosové funkcie a PI regulátory

u_{pb}	Prenosová funkcia	y_{pb}	Δy_{pb} pri $u_{pb} \pm 0.5$ V	PI parametre
2 V	$\frac{5.79}{s^2 + 44.21s + 7.276}$	4.4884	± 0.41	$K_p = 2.77, K_i = 1.48$
4 V	$\frac{6.074}{s^2 + 59.01s + 8.95}$	6.3412	± 0.35	$K_p = 3.58, K_i = 1.84$
6 V	$\frac{7.161}{s^2 + 52.87s + 21.37}$	7.7441	± 0.18	$K_p = 1.83, K_i = 2.20$

Keďže pôvodné statické charakteristiky systému boli merané ešte počas letného obdobia, bolo rozhodnuté hodnoty y_{pb} pre tieto experimenty zmerať znova, rovnakým spôsobom ako v sekcii 2.1.2. V tabuľke sú preto uvedené aktuálne hodnoty pracovných bodov, ktoré vernejšie reprezentujú stav systému počas meraní v zimnom období.

4.1 Opis a schéma riadenia s použitím prepínača

V tejto časti je uvažovaná riadiaca štruktúra, ktorá pozostáva z dvoch PI regulátorov. Prvý z nich predstavuje globálny PI regulátor, ktorý pracuje s absolútnymi veličinami systému. Druhý predstavuje lokálny PI regulátor, ktorý pracuje s normalizovanými veličinami v okolí aktuálneho pracovného bodu.

Lokálny regulátor využíva normalizované premenné

$$y_N = y - y_{pb}, \quad u_N = u - u_{pb}, \quad (54)$$

kde y_{pb} a u_{pb} predstavujú výstupnú a vstupnú hodnotu príslušného pracovného bodu. Parametre lokálneho PI regulátora sú volené pomocou princípu gain scheduling, pričom sa používajú hodnoty uvedené v tabuľke 2.

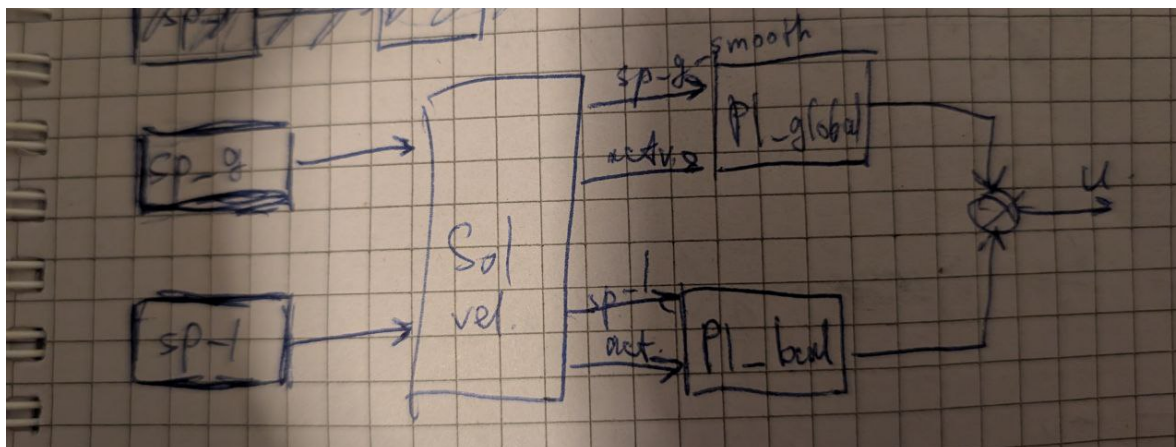
Globálny regulátor pracuje s absolútnou žiadanou hodnotou. Aby prechod medzi pracovnými bodmi neprebíhal príliš prudko, globálna žiadaná hodnota je pred vstupom do regulátora filtrovaná prenosom druhého rádu v tvare

$$G_f(s) = \frac{1}{(10s + 1)^2}. \quad (55)$$

Takto zvolený filter zabezpečuje dostatočne plynulý priebeh globálnej žiadanej hodnoty, v dôsledku čoho prechod medzi pracovnými bodmi prebieha pomalšie a s menším rizikom vzniku neželaných prechodových javov.

O tom, ktorý PI regulátor bude v danom okamihu aktívny, rozhoduje blok logiky prepínača. Jeho úloha je dvojité. Po prvé určuje, či má byť aktívny globálny alebo lokálny regulátor. Po druhé spracúva globálnu žiadanú hodnotu a vytvára jej filtrovanú verziu pre globálnu reguláciu.

Navrhnutá bloková schéma algoritmu je znázornená na obr. 21.



Obr. 21: Bloková schéma riadenia s globálnym a lokálnym PI regulátorom

Logika prepínania je založená na nasledujúcom princípe. Ak je detegovaná skoková zmena globálnej žiadanej hodnoty, aktivuje sa globálny PI regulátor, ktorý zabezpečuje prechod medzi pracovnými bodmi. Ak globálna regulácia nie je aktívna, systém sa automaticky prepne do režimu lokálnej regulácie, v ktorom sa používa PI regulátor naladený pre aktuálny pracovný bod.

V ďalšej časti budú predstavené dve konkrétne realizácie tejto logiky. Budú sa líšiť predovšetkým spôsobom rozhodovania o tom, kedy sa globálna regulácia ukončí a kedy sa aktivuje lokálna regulácia.

Sledovanie integrátora

Dôležitou požiadavkou navrhnutej štruktúry je zabezpečenie plynulého prepínania medzi globálnym a lokálnym PI regulátorom. Ak by sa po deaktivácii jedného regulátora jeho integračná zložka nechala bez aktualizácie, po opätovnom aktivovaní by mohla mať hodnotu, ktorá nezodpovedá aktuálne realizovanému akčnému zásahu systému. To by viedlo ku skokovej zmene riadenia a k neželanej dynamike pri prepnutí.

Z tohto dôvodu je pri neaktívnom regulátore použitý jednoduchý sledovací mechanizmus integrátora. Jeho úlohou je priebežne nastavovať integračný stav tak, aby po aktivovaní regulátora nedošlo k skokovej zmene jeho výstupu. V prípade lokálneho regulátora sa pri tomto výpočte využíva aj hodnota u_{pb} , ktorá je počas globálnej regulácie zvolená podľa aktuálne uvažovaného pracovného bodu z tabuľky 2. Takto sa dosiahne plynulé prepnutie medzi oboma regulačnými vetvami.

Zákon riadenia

Označme binárne aktivačné signály lokálneho a globálneho regulátora ako a_l a a_g , pričom platí

$$a_l, a_g \in \{0, 1\}, \quad a_l + a_g = 1. \quad (56)$$

Chyba lokálneho regulátora je definovaná vzťahom

$$e_l = r_l - (y - y_{pb}), \quad (57)$$

kde r_l predstavuje lokálnu žiadanú hodnotu.

Filtrovaná globálna žiadaná hodnota je označená ako $r_{g,f}$, pričom chyba globálneho regulátora je

$$e_g = r_{g,f} - y. \quad (58)$$

Lokálny PI regulátor má tvar

$$u_l = (u_{pb} + K_{p,l}e_l + K_{i,l}I_l) a_l, \quad (59)$$

a globálny PI regulátor je definovaný vzťahom

$$u_g = (K_{p,g}e_g + K_{i,g}I_g) a_g. \quad (60)$$

Celkový akčný zásah je daný súčtom oboch vetiev

$$u = u_l + u_g. \quad (61)$$

Po dosadení (59) a (60) do (61) dostávame rozvinutý tvar zákona riadenia

$$u = (u_{pb} + K_{p,l}e_l + K_{i,l}I_l) a_l + (K_{p,g}e_g + K_{i,g}I_g) a_g. \quad (62)$$

Keďže v každom okamihu je aktívna práve jedna regulačná vetva, možno zákon riadenia zapísať aj v tvare

$$u = \begin{cases} u_{pb} + K_{p,l}e_l + K_{i,l}I_l, & a_l = 1, a_g = 0, \\ K_{p,g}e_g + K_{i,g}I_g, & a_l = 0, a_g = 1. \end{cases} \quad (63)$$

Pri aktívnom regulátore sa integračný stav aktualizuje štandardne na základe regulačnej chyby. Pri neaktívnom regulátore sa integrátor priamo dopočíta z aktuálneho akčného zásahu, aby bol pripravený na plynulé prevzatie riadenia.

Pre lokálny regulátor sa pri neaktívnom stave používa vzťah

$$I_l = \frac{(u - u_{pb}) - K_{p,l}e_l}{K_{i,l}}, \quad a_l = 0, \quad (64)$$

a pre globálny regulátor analogicky

$$I_g = \frac{u - K_{p,g}e_g}{K_{i,g}}, \quad a_g = 0. \quad (65)$$

Takto definovaný spôsob sledovania integrátora zabezpečuje, že pri prepnutí medzi lokálnou a globálnou regulačnou vetvou nedochádza k skokovej zmene akčného zásahu.

4.1.1 Globálna regulácia po určitý čas

Prvou uvažovanou realizáciou prepínacej logiky je regulácia založená na pevne zvolenom čase pôsobenia globálneho PI regulátora. Predpokladá sa pritom, že na prechod z jedného pracovného bodu do druhého postačuje približne konštantný čas. Na základe experimentálnych pozorovaní bol tento čas určený ako

$$T_{sw} = 80 \text{ s}. \quad (66)$$

Pri skokovej zmene globálnej žiadanej hodnoty sa teda aktivuje globálny PI regulátor, ktorý zostáva aktívny počas intervalu T_{sw} . Po uplynutí tohto času sa globálna vetva deaktivuje a riadenie automaticky preberá lokálny PI regulátor. Pri lokálnej regulácii sa používajú hodnoty u_{pb} , y_{pb} a parametre PI regulátora z tabuľky 2.

Filter globálnej žiadanej hodnoty z rovnice (55) je pre potreby diskkrétnej implementácie vhodné previesť do stavového priestoru. Pre prenos

$$G_f(s) = \frac{R_{g,f}(s)}{R_g(s)} = \frac{1}{(10s+1)^2} = \frac{0.01}{s^2 + 0.2s + 0.01} \quad (67)$$

zvolíme stavový vektor

$$x_f(t) = \begin{bmatrix} x_{f1}(t) \\ x_{f2}(t) \end{bmatrix}. \quad (68)$$

V kanonickom tvare má filter podobu

$$\dot{x}_f(t) = A_f x_f(t) + b_f r_g(t), \quad (69)$$

$$r_{g,f}(t) = c_f^T x_f(t), \quad (70)$$

kde

$$A_f = \begin{bmatrix} 0 & 1 \\ -0.01 & -0.2 \end{bmatrix}, \quad b_f = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_f = \begin{bmatrix} 0.01 \\ 0 \end{bmatrix}. \quad (71)$$

Pri Eulerovej diskretizácii so vzorkovacou periódou T_s má stavová rovnica tvar

$$x_f(k+1) = x_f(k) + T_s (A_f x_f(k) + b_f r_g(k)), \quad (72)$$

a výstup filtra je určený vzťahom

$$r_{g,f}(k) = c_f^T x_f(k). \quad (73)$$

Z implementačného hľadiska je preto potrebné prenášať medzi jednotlivými krokmi stav filtra globálnej žiadanej hodnoty, integračné stavy oboch PI regulátorov a predchádzajúcu hodnotu globálnej žiadanej hodnoty. V prípade časovo riadeného prepínania je zároveň potrebné poznať čas aktivácie globálnej regulačnej vetvy, pričom aktuálny čas je dostupný ako vstup algoritmu. Okrem toho je potrebné mať k dispozícii tabuľkové hodnoty pracovných bodov y_{pb} , u_{pb} a príslušné parametre lokálnych PI regulátorov, keďže lokálna regulačná vetva je realizovaná s využitím gain scheduling.

Pseudokód tejto realizácie je uvedený nižšie.

```

1  % Inputs:
2  % t(k)   - current time
3  % r_g(k) - global reference
4  % r_l(k) - local reference
5  % y(k)   - plant output
6  %
7  % States:
8  % x_f(2x1) - state of global reference filter
9  % I_l      - integral state of local PI controller
10 % I_g      - integral state of global PI controller
11 % t_g_start - activation time of global mode
12 % r_g_prev - previous global reference
13 % active_g - activity flag of global controller
14 %
15 % Parameters:
16 % Ts, T_sw
17 % Kp_g, Ki_g - global PI parameters
18 %
19 % Tables:
20 % y_pb_table, u_pb_table
21 % Kp_l_table, Ki_l_table
22 %
23 % Filter matrices
24 A_f = [ 0, 1;
25         -0.01, -0.2 ];
26
27 b_f = [ 0;
28         1 ];
29
30 c_f = [ 0.01, 0 ];
31

```



```

32 % -----
33 % 1. Detect step change of global reference
34 % -----
35 if r_g ~= r_g_prev
36     active_g = 1;
37     t_g_start = t;
38 end
39
40 % -----
41 % 2. Filter global reference
42 % -----
43 r_gf = c_f * x_f;
44 x_f_next = x_f + Ts * ( A_f * x_f + b_f * r_g );
45
46 % -----
47 % 3. Select local operating point and PI parameters from table
48 % -----
49 select y_pb, u_pb, Kp_l, Ki_l from table according to current operating point
50
51 % -----
52 % 4. Switching logic
53 % -----
54 if active_g == 1
55     if ( t - t_g_start ) >= T_sw
56         active_g = 0;
57     end
58 end
59
60 active_l = 1 - active_g;
61
62 % -----
63 % 5. Control errors
64 % -----
65 e_l = r_l - ( y - y_pb );
66 e_g = r_gf - y;
67
68 % -----
69 % 6. Control law
70 % -----
71 u_l = active_l * ( u_pb + Kp_l * e_l + Ki_l * I_l );
72 u_g = active_g * ( Kp_g * e_g + Ki_g * I_g );
73 u = u_l + u_g;
74
75 % -----
76 % 7. Integral states update
77 % -----
78 if active_l == 1
79     I_l_next = I_l + Ts * e_l;
80 else
81     I_l_next = ( ( u - u_pb ) - Kp_l * e_l ) / Ki_l;
82 end
83
84 if active_g == 1
85     I_g_next = I_g + Ts * e_g;
86 else
87     I_g_next = ( u - Kp_g * e_g ) / Ki_g;
88 end
89
90 % -----
91 % 8. Memory update
92 % -----
93 r_g_prev_next = r_g;
94 t_g_start_next = t_g_start;

```

Výpis kódu 5: Pseudokód prepínania s globálnou reguláciou po určitý čas

Uvedený spôsob prepínania je založený na predpoklade, že globálny PI regulátor potrebuje na presun systému do nového pracovného bodu približne konštantný čas. Takýto prístup je implementačne jednoduchý, avšak jeho nevýhodou je, že dynamika systému nie je v reálnych podmienkach úplne konštantná. Mení sa okrem iného aj v závislosti od aktuálnych podmienok okolia, napríklad od teploty v miestnosti. To

znamená, že hodnoty u_{pb} a y_{pb} uvedené v tabuľke 2 nemusia v danom experimente presne zodpovedať skutočnému ustálenému stavu systému.

Rovnako nie je zaručené, že za pevne stanovený čas 80 s sa systém vždy ustáli dostatočne blízko požadovaného pracovného bodu. Z tohto dôvodu bol navrhnutý aj druhý spôsob prepínania, ktorý je založený priamo na detekcii ustálenia.

4.1.2 Prepínanie na základe detekcie ustálenia

Druhá uvažovaná realizácia prepínacej logiky nie je založená na predpoklade, že ustálenie nastane po vopred známom čase. Namiesto toho sa globálna regulácia ukončí až vtedy, keď sa na základe meraných údajov zistí, že systém sa skutočne ustálil.

V tejto štruktúre má globálny PI regulátor úlohu pomalého a robustného regulátora, ktorý zabezpečuje prechod medzi pracovnými bodmi v rámci celého pracovného rozsahu systému. Lokálny PI regulátor má naopak za úlohu presnú reguláciu v okolí aktuálneho pracovného bodu, teda v oblasti približne $u_{pb} \pm 0.5$ V. Z tohto dôvodu pracuje s normalizovanými veličinami a jeho parametre sa volia pomocou gain scheduling na základe tabuľkových hodnôt pracovných bodov.

Pri tejto metóde sa okrem stavu filtra, integračných stavov a predchádzajúcej hodnoty globálnej žiadanej hodnoty uchováva aj posledných 30 hodnôt výstupu y a posledných 30 hodnôt vstupu u . Globálna regulácia sa opäť aktivuje pri skoku globálnej žiadanej hodnoty, avšak jej deaktivácia nastane až po splnení podmienky ustálenia.

Za ustálený stav sa v tomto prípade považuje situácia, keď sa aspoň 25 z posledných 30 hodnôt výstupu nachádza v intervale

$$\langle r_g - 0.07 r_g, r_g + 0.07 r_g \rangle. \quad (74)$$

Po splnení tejto podmienky sa ako aktuálna hodnota pracovného bodu určí aritmetický priemer posledných 30 hodnôt výstupu a vstupu, teda

$$y_{pb}^* = \frac{1}{30} \sum_{i=1}^{30} y_i, \quad u_{pb}^* = \frac{1}{30} \sum_{i=1}^{30} u_i. \quad (75)$$

Takto získané hodnoty reprezentujú odhad skutočného pracovného bodu, ktorý bol dosiahnutý globálnou reguláciou. Následne sa vyberú parametre lokálneho PI regulátora podľa toho, ku ktorému pracovnému bodu z tabuľky 2 je hodnota y_{pb}^* najbližšia.

Pseudokód tejto realizácie je uvedený nižšie.

```

1  % Inputs:
2  % t(k)    - current time
3  % r_g(k)  - global reference
4  % r_l(k)  - local reference
5  % y(k)    - plant output
6  %
7  % States:
8  % x_f(2x1) - state of global reference filter
9  % I_l      - integral state of local PI controller
10 % I_g      - integral state of global PI controller
11 % y_hist(30) - history of output values
12 % u_hist(30) - history of input values
13 % r_g_prev - previous global reference
14 % active_g - activity flag of global controller
15 %
16 % Parameters:
17 % Ts
18 % Kp_g, Ki_g - global PI parameters
19 %
20 % Tables:
21 % y_pb_table, u_pb_table
22 % Kp_l_table, Ki_l_table
23 %
24 % Filter matrices
25 A_f = [ 0,    1;
26         -0.01, -0.2 ];

```

```

27
28 b_f = [ 0;
29         1 ];
30
31 c_f = [ 0.01, 0 ];
32
33 % -----
34 % 1. Detect step change of global reference
35 % -----
36 if r_g ~= r_g_prev
37     active_g = 1;
38 end
39
40 % -----
41 % 2. Filter global reference
42 % -----
43 r_gf = c_f * x_f;
44 x_f_next = x_f + Ts * ( A_f * x_f + b_f * r_g );
45
46 % -----
47 % 3. Update histories
48 % -----
49 shift y_hist and append y(k)
50 shift u_hist and append u(k)
51
52 % -----
53 % 4. Select local operating point and PI parameters from table
54 % -----
55 select y_pb, u_pb, Kp_l, Ki_l from table according to current operating point
56
57 % -----
58 % 5. Control errors
59 % -----
60 e_l = r_l - ( y - y_pb );
61 e_g = r_gf - y;
62
63 % -----
64 % 6. Control law
65 % -----
66 u_l = (1 - active_g) * ( u_pb + Kp_l * e_l + Ki_l * I_l );
67 u_g = active_g * ( Kp_g * e_g + Ki_g * I_g );
68 u = u_l + u_g;
69
70 % -----
71 % 7. Settling detection
72 % -----
73 count_ok = number of elements in y_hist within r_g +/- 7%
74
75 if active_g == 1 && count_ok >= 25
76     y_pb_est = mean( y_hist );
77     u_pb_est = mean( u_hist );
78
79     select local PI parameters corresponding to nearest tabulated y_pb
80     set y_pb = y_pb_est;
81     set u_pb = u_pb_est;
82
83     active_g = 0;
84 end
85
86 active_l = 1 - active_g;
87
88 % -----
89 % 8. Integral states update
90 % -----
91 if active_l == 1
92     I_l_next = I_l + Ts * e_l;
93 else
94     I_l_next = ( (u - u_pb) - Kp_l * e_l ) / Ki_l;
95 end
96
97 if active_g == 1
98     I_g_next = I_g + Ts * e_g;

```

```

99 else
100     I_g_next = ( u - Kp_g * e_g ) / Ki_g;
101 end
102
103 % -----
104 % 9. Memory update
105 % -----
106 r_g_prev_next = r_g;

```

Výpis kódu 6: Pseudokód prepínania na základe detekcie ustálenia

Táto metóda je z výpočtového hľadiska zložitejšia, pretože vyžaduje uchovávanie histórie signálov a priebežné vyhodnocovanie podmienky ustálenia. Na druhej strane je však podstatne robustnejšia, pretože nepredpokladá fixný čas ustálenia, ale vychádza zo skutočného správania systému.

Keďže perióda regulácie je $T_s = 0.1$ s, čo predstavuje z pohľadu použitého hardvéru dostatočne veľký čas na vykonanie potrebných výpočtov, bolo rozhodnuté implementovať práve tento prístup. V porovnaní s časovo riadeným prepínaním poskytuje spoľahlivejšie ukončenie globálnej regulácie a lepšie zohľadňuje zmeny dynamiky systému spôsobené reálnymi prevádzkovými podmienkami.

5 MRAC regulácia – gradientná metóda

V tejto kapitole je riešená adaptívna regulácia typu MRAC (Model Reference Adaptive Control), ktorá umožňuje automatické prispôsobovanie parametrov regulátora pri nepresne známej dynamike riadeného systému. Na rozdiel od klasických regulátorov s pevnými parametrami MRAC využíva referenčný model, ktorý definuje požadované správanie uzavretého regulačného obvodu, a adaptačný mechanizmus, ktorý zabezpečuje sledovanie tohto správania v čase.

V ďalšej časti je popísaný samotný algoritmus gradientnej MRAC regulácie, vrátane voľby referenčného modelu, štruktúry zákona riadenia a adaptačného zákona založeného na gradientnej metóde.

5.1 Popis algoritmu

Gradientná MRAC regulácia je založená na myšlienke, že dynamika riadeného systému má byť v čase prispôbovaná tak, aby výstup systému sledoval výstup zvoleného referenčného modelu. Návrh algoritmu pozostáva z troch základných častí: voľba referenčného modelu, návrh zákona riadenia a odvodenie adaptačného zákona.

Model riadeného systému

Uvažujme lineárny spojitý systém druhého rádu

$$G_s(s) = \frac{b_0}{s^2 + a_1 s + a_0}, \quad (76)$$

kde a_0 , a_1 , b_0 sú neznáme konštanty. Predpokladá sa znalosť znamienka parametra b_0 .

Referenčný model a statické zosilnenie

Pri voľbe referenčného modelu je potrebné zabezpečiť, aby jeho statické zosilnenie bolo rovné jednej, t. j.

$$K = 1,$$

aby nedochádzalo k nechcenému zosilňovaniu referenčného signálu.

Pre systém druhého rádu zvolíme referenčný model

$$\frac{y_m(s)}{r(s)} = \frac{b_{0m}}{s^2 + a_{1m}s + a_{0m}}, \quad (77)$$

pričom sa volí

$$b_{0m} = a_{0m}, \quad (78)$$

čím sa zabezpečí jednotkové statické zosilnenie modelu.

Zákon riadenia

Všeobecný tvar zákona riadenia pri splnenej podmienke zhody je

$$u(t) = \Theta^T(t) \omega(t), \quad (79)$$

kde regresný vektor obsahuje merateľné signály.

Pre systém druhého rádu zvolíme

$$\omega(t) = \begin{bmatrix} y(t) \\ \dot{y}(t) \\ r(t) \end{bmatrix}. \quad (80)$$

Zákon riadenia potom nadobúda tvar

$$u(t) = \Theta_1 y(t) + \Theta_2 \dot{y}(t) + \Theta_3 r(t). \quad (81)$$

Podmienka zhody a ideálne parametre

Po dosadení (81) do systému (76) dostávame podmienku zhody

$$y(s) (s^2 + (a_1 - b_0\Theta_2)s + a_0 - b_0\Theta_1) = b_0\Theta_3 r(s). \quad (82)$$

Prenos uzavretého regulačného obvodu je

$$\frac{y(s)}{r(s)} = \frac{b_0\Theta_3}{s^2 + (a_1 - b_0\Theta_2)s + (a_0 - b_0\Theta_1)}. \quad (83)$$

Porovnaním s referenčným modelom získame ideálny vektor parametrov

$$\Theta_1^* = \frac{a_0 - a_{0m}}{b_0}, \quad (84)$$

$$\Theta_2^* = \frac{a_1 - a_{1m}}{b_0}, \quad (85)$$

$$\Theta_3^* = \frac{b_{0m}}{b_0}. \quad (86)$$

Adaptačná odchýlka

Adaptačná odchýlka je definovaná ako

$$e(t) = y(t) - y_m(t), \quad (87)$$

pričom cieľom regulácie je dosiahnuť

$$\lim_{t \rightarrow \infty} e(t) = 0. \quad (88)$$

Účelová funkcia a MIT pravidlo

Minimalizovaná je účelová funkcia

$$J(\Theta) = \frac{1}{2} e^2(t). \quad (89)$$

MIT pravidlo je formulované ako

$$\frac{d\Theta}{dt} = -\alpha \frac{\partial J}{\partial \Theta} = -\alpha e \frac{\partial e}{\partial \Theta}. \quad (90)$$

Keďže y_m nezávisí od Θ , platí

$$\frac{d\Theta}{dt} = -\alpha e \frac{\partial y}{\partial \Theta}. \quad (91)$$

Zákon adaptácie pre systém druhého rádu

Pre systém druhého rádu sú adaptačné zákony dané vzťahmi

$$\dot{\Theta}_1 = -\alpha_1 e \operatorname{sign}(b_0) \frac{1}{s^2 + a_{1m}s + a_{0m}} y, \quad (92)$$

$$\dot{\Theta}_2 = -\alpha_2 e \operatorname{sign}(b_0) \frac{s}{s^2 + a_{1m}s + a_{0m}} y, \quad (93)$$

$$\dot{\Theta}_3 = -\alpha_3 e \operatorname{sign}(b_0) \frac{1}{s^2 + a_{1m}s + a_{0m}} r. \quad (94)$$

Člen s v adaptačnom zákone pre parameter Θ_2 zohľadňuje dynamický vplyv derivácie výstupu v zákone riadenia.

5.1.1 Prevod systému do stavového priestoru

Pre potreby diskkrétnej implementácie je vhodné preniesť opis riadeného systému z tvaru prenosovej funkcie do tvaru stavového modelu.

Uvažujme všeobecný lineárny systém druhého rádu

$$G_s(s) = \frac{Y(s)}{U(s)} = \frac{b_1 s + b_0}{s^2 + a_1 s + a_0}. \quad (95)$$

Tento prenos je možné interpretovať ako kaskádové spojenie dvoch čiastkových prenosov zavedením pomocnej premennej $Z(s)$

$$\frac{Z(s)}{U(s)} = \frac{1}{s^2 + a_1 s + a_0}, \quad (96)$$

$$\frac{Y(s)}{Z(s)} = b_1 s + b_0. \quad (97)$$

Platí teda

$$\frac{Y(s)}{U(s)} = \frac{Y(s)}{Z(s)} \frac{Z(s)}{U(s)}. \quad (98)$$

Prvý z uvedených prenosov (96) možno zapísať v časovej oblasti ako diferenciálnu rovnicu

$$\ddot{z}(t) + a_1 \dot{z}(t) + a_0 z(t) = u(t). \quad (99)$$

Táto rovnica je druhého rádu a je možné ju previesť na sústavu diferenciálnych rovníc prvého rádu vhodnou voľbou stavových veličín.

Zavedme prvú stavovú veličinu

$$x_1(t) = z(t), \quad (100)$$

z čoho vyplýva

$$\dot{x}_1(t) = \dot{z}(t). \quad (101)$$

Ako druhú stavovú veličinu zvolíme

$$x_2(t) = \dot{z}(t), \quad (102)$$

a teda

$$\dot{x}_2(t) = \ddot{z}(t). \quad (103)$$

Z rovnice (99) je možné vyjadriť druhú deriváciu

$$\ddot{z}(t) = -a_1 \dot{z}(t) - a_0 z(t) + u(t), \quad (104)$$

čo po dosadení za stavové premenné vedie k

$$\dot{x}_2(t) = -a_1 x_2(t) - a_0 x_1(t) + u(t). \quad (105)$$

Sústava diferenciálnych rovníc prvého rádu má teda tvar

$$\dot{x}_1(t) = x_2(t), \quad (106)$$

$$\dot{x}_2(t) = -a_1 x_2(t) - a_0 x_1(t) + u(t). \quad (107)$$

V maticovom zápise možno systém vyjadriť ako

$$\dot{x}(t) = \begin{bmatrix} 0 & 1 \\ -a_0 & -a_1 \end{bmatrix} x(t) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(t), \quad (108)$$

kde stavový vektor je definovaný ako

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}. \quad (109)$$

Výstupná rovnica vyplýva z prenosu (97). V časovej oblasti platí

$$y(t) = b_1 \dot{z}(t) + b_0 z(t). \quad (110)$$

Po vyjadrení pomocou stavových veličín dostávame

$$y(t) = b_1 x_2(t) + b_0 x_1(t), \quad (111)$$

resp. v maticovom tvare

$$y(t) = \begin{bmatrix} b_0 & b_1 \end{bmatrix} x(t). \quad (112)$$

Celý systém je teda v štandardnom stavovom tvare

$$\dot{x}(t) = Ax(t) + bu(t), \quad (113)$$

$$y(t) = c^T x(t), \quad (114)$$

Celý systém je teda v štandardnom stavovom tvare

$$\dot{x}(t) = Ax(t) + bu(t), \quad (115)$$

$$y(t) = c^T x(t), \quad (116)$$

kde jednotlivé matice majú tvar

$$A = \begin{bmatrix} 0 & 1 \\ -a_0 & -a_1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c = \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}. \quad (117)$$

Zavedená pomocná premenná $z(t)$ predstavuje interný stav systému, ktorý bol použitý na rozklad prenosovej funkcie do kaskádového tvaru. Pri zvolenej realizácii však výstup systému vzniká priamo ako lineárna kombinácia premenných $z(t)$ a $\dot{z}(t)$.

Z tohto dôvodu možno bez straty všeobecnosti interpretovať stavový vektor ako

$$x(t) = \begin{bmatrix} y(t) \\ \dot{y}(t) \end{bmatrix}, \quad (118)$$

keďže prvá stavová veličina reprezentuje samotný výstup systému a druhá jeho časovú deriváciu. Táto voľba je prirodzená pre systém druhého rádu a priamo nadväzuje na štruktúru diferenciálnej rovnice.

Stavový opis referenčného modelu

Referenčný model definovaný prenosom (77) je potrebné previesť do stavového tvaru.

Zvolíme stavový vektor

$$x_{ym}(t) = \begin{bmatrix} y_m(t) \\ \dot{y}_m(t) \end{bmatrix}. \quad (119)$$

Na základe všeobecného tvaru odvodeného v predchádzajúcej časti má referenčný model podobu

$$\dot{x}_{ym}(t) = A_{ym}x_{ym}(t) + b_{ym}r(t), \quad (120)$$

$$y_m(t) = c_{ym}^T x_{ym}(t), \quad (121)$$

kde

$$A_{ym} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{ym} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{ym} = \begin{bmatrix} b_{0m} \\ 0 \end{bmatrix}. \quad (122)$$

Stavový opis filtra výstupu

Filtračný člen použitý v adaptačnom zákone pre parameter Θ_1 (92) je daný prenosom

$$\frac{y_f(s)}{y(s)} = \frac{1}{s^2 + a_{1m}s + a_{0m}}. \quad (123)$$

Zvolíme stavový vektor

$$x_{yf}(t) = \begin{bmatrix} y_f(t) \\ \dot{y}_f(t) \end{bmatrix}. \quad (124)$$

Stavový opis má tvar

$$\dot{x}_{yf}(t) = A_{yf}x_{yf}(t) + b_{yf}y(t), \quad (125)$$

$$y_f(t) = c_{yf}^T x_{yf}(t), \quad (126)$$

kde

$$A_{yf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{yf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{yf} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}. \quad (127)$$

Stavový opis filtra derivácie výstupu

V adaptačnom zákone pre parameter Θ_2 (93) sa nachádza člen

$$\frac{\dot{y}_f(s)}{y(s)} = \frac{s}{s^2 + a_{1m}s + a_{0m}}, \quad (128)$$

ktorý zodpovedá derivácii výstupu predchádzajúceho filtra.

Použijeme rovnaký stavový vektor $x_{yf}(t)$, pričom výstupná rovnica reprezentuje druhú stavovú veličinu

$$\dot{x}_{yf}(t) = A_{ydf}x_{yf}(t) + b_{ydf}y(t), \quad (129)$$

$$\dot{y}_f(t) = c_{ydf}^T x_{yf}(t), \quad (130)$$

kde

$$A_{ydf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{ydf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{ydf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \quad (131)$$

Stavový opis filtra referencie

Filtračný člen použitý v adaptačnom zákone pre parameter Θ_3 (94) je daný prenosom

$$\frac{r_f(s)}{r(s)} = \frac{1}{s^2 + a_{1m}s + a_{0m}}. \quad (132)$$

Zvolíme stavový vektor

$$x_{rf}(t) = \begin{bmatrix} r_f(t) \\ \dot{r}_f(t) \end{bmatrix}. \quad (133)$$

Stavový opis má tvar

$$\dot{x}_{rf}(t) = A_{rf}x_{rf}(t) + b_{rf}r(t), \quad (134)$$

$$r_f(t) = c_{rf}^T x_{rf}(t), \quad (135)$$

kde

$$A_{rf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{rf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{rf} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}. \quad (136)$$

Zhrnutie štruktúry dynamických blokov

Zo získaných stavových opisov vyplýva, že všetky dynamické bloky využívajú rovnakú menovateľovú dynamiku, teda

$$A_{ym} = A_{yf} = A_{ydf} = A_{rf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad (137)$$

a rovnako

$$b_{ym} = b_{yf} = b_{ydf} = b_{rf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \quad (138)$$

Jednotlivé bloky sa odlišujú iba výstupnou maticou c .

Pre filter výstupu a jeho derivácie možno zapísať spoločnú výstupnú rovnicu v maticovom tvare

$$\begin{bmatrix} y_f(t) \\ \dot{y}_f(t) \end{bmatrix} = C_{yf} x_{yf}(t), \quad (139)$$

kde

$$C_{yf} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \quad (140)$$

Pre filter referencie platí

$$r_f(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} x_{rf}(t). \quad (141)$$

Referenčný model je realizovaný samostatne so stavovým vektorom $x_{ym}(t)$, pričom jeho výstup je

$$y_m(t) = \begin{bmatrix} b_{0m} & 0 \end{bmatrix} x_{ym}(t). \quad (142)$$

5.1.2 Pseudokód gradientnej MRAC regulácie (diskrétna realizácia)

Diskrétna realizácia gradientnej MRAC regulácie je vykonávaná v pevnom vzorkovacom intervale T_s . V každom vzorkovacom kroku sú k dispozícii aktuálne merané veličiny a stavové premenné uložené z predchádzajúceho kroku.

Keďže regulátor MRAC je navrhnutý pre linearizovaný model v okolí pracovného bodu, samotnej adaptácii predchádza fáza ustálenia. Počas tejto fázy je na vstup systému privádzaný konštantný riadiaci zásah u_0 po dobu T_{settle} , aby sa systém dostal do okolia zvoleného pracovného bodu. Po ukončení tejto fázy sú uložené ustálené hodnoty výstupu y_0 a referencie r_0 a ďalšia činnosť regulátora prebieha s normalizovanými veličinami definovanými vzhľadom na pracovný bod

$$\tilde{y}(k) = y(k) - y_0, \quad \tilde{r}(k) = r(k) - r_0, \quad \tilde{u}(k) = u(k) - u_0. \quad (143)$$

Zo systému je dostupný iba výstup $y(k)$. Keďže regulačný zákon využíva aj jeho časovú deriváciu, je potrebné túto veličinu numericky aproximovať. V diskretnom čase je derivácia normalizovaného výstupu určená pomocou spätnej diferencie

$$\dot{\tilde{y}}(k) = \frac{\tilde{y}(k) - \tilde{y}(k-1)}{T_s}, \quad (144)$$

kde $\tilde{y}(k-1)$ predstavuje hodnotu normalizovaného výstupu z predchádzajúceho vzorkovacieho kroku.

Na základe regresného vektora

$$\tilde{\omega}(k) = \begin{bmatrix} \tilde{y}(k) \\ \dot{\tilde{y}}(k) \\ \tilde{r}(k) \end{bmatrix} \quad (145)$$

a aktuálneho vektora parametrov $\Theta(k)$ je po skončení fázy ustálenia vypočítaná adaptívna zložka riadiaceho zásahu

$$\tilde{u}(k) = \Theta^T(k)\tilde{\omega}(k). \quad (146)$$

Celkový riadiaci zásah má potom tvar

$$u(k) = u_0 + \tilde{u}(k). \quad (147)$$

Následne je určená chyba sledovania

$$e(k) = \tilde{y}(k) - y_m(k), \quad (148)$$

kde $y_m(k)$ je výstup referenčného modelu budeného normalizovanou referenciou $\tilde{r}(k)$.

Adaptačný zákon MIT pravidla je realizovaný v diskretnom tvare pomocou Eulerovej integrácie

$$\Theta(k+1) = \Theta(k) + T_s \dot{\Theta}(k), \quad (149)$$

pričom $\dot{\Theta}(k)$ vyplýva zo vzťahov (92) – (94) a obsahuje filtrované veličiny $y_f(k)$, $\dot{y}_f(k)$ a $r_f(k)$ získané z normalizovaných signálov.

Stavové rovnice referenčného modelu a jednotlivých filtračných blokov sú integrované rovnakým spôsobom v tvare

$$x(k+1) = x(k) + T_s(Ax(k) + bv(k)), \quad (150)$$

kde vstup $v(k)$ predstavuje normalizovanú referenciu $\tilde{r}(k)$ alebo normalizovaný výstup $\tilde{y}(k)$ v závislosti od konkrétneho bloku.

Počas fázy ustálenia sa adaptácia nevykonáva a stavy referenčného modelu aj filtračných blokov zostávajú nezmenené. Po vykonaní všetkých aktualizácií je hodnota $y(k)$ uložená pre použitie v nasledujúcom kroku pri výpočte numerickej derivácie.

Pseudokód diskretizovanej realizácie algoritmu je uvedený nižšie.

```

1
2 % Inputs:
3 % r(k) - reference signal
4 % y(k) - plant output
5 % t(k) - current time
6 %
7 % States:
8 % x_ym(2x1) - states of reference model
9 % x_yf(2x1) - states of output filter
10 % x_rf(2x1) - states of reference filter
11 % Theta(3x1) - [Theta1; Theta2; Theta3]
12 % y_prev - previous plant output
13 % y0 - stored operating-point output
14 % r0 - stored operating-point reference
15 % sw - switching flag
16 %
17 % Parameters:
18 % a0m, a1m, b0m
19 % alpha1, alpha2, alpha3
20 % sign_b0
21 % Ts
22 % u0
23 % T_settle
24
25 % Common dynamics
26 A = [ 0, 1;
27       -a0m, -a1m ];
28
29 b = [ 0;
30       1 ];
31
32 c_yf = [ 1, 0;
33          0, 1 ];
34
35 c_rf = [ 1, 0 ];
36
```

```

37 c_ym = [ b0m, 0 ];
38
39 % Default memory update
40 y_prev_next = y;
41 y0_next = y0;
42 r0_next = r0;
43 sw_next = sw;
44
45 if t(k) < T_settle
46
47     % Settling phase
48     u(k) = u0;
49
50     % Freeze adaptation and dynamic blocks
51     Theta_next = Theta;
52     x_ym_next = x_ym;
53     x_yf_next = x_yf;
54     x_rf_next = x_rf;
55
56     ym(k) = c_ym * x_ym;
57
58 else
59
60     % Store operating point only once
61     if sw < 0.5
62         y0_next = y(k);
63         r0_next = r(k);
64         sw_next = 1;
65     end
66
67     % Normalized signals
68     y_tilde = y(k) - y0_next;
69     r_tilde = r(k) - r0_next;
70     y_prev_tilde = y_prev - y0_next;
71
72     % Differentiation
73     ydot_tilde = ( y_tilde - y_prev_tilde ) / Ts;
74
75     % Outputs from current states
76     ym = c_ym * x_ym;
77     yf_vec = c_yf * x_yf;
78     yf = yf_vec(1);
79     ydot_f = yf_vec(2);
80     rf = c_rf * x_rf;
81
82     % Tracking error
83     e = y_tilde - ym;
84
85     % Control law
86     omega = [ y_tilde; ydot_tilde; r_tilde ];
87     u_tilde = Theta.' * omega;
88     u = u0 + u_tilde;
89
90     % MIT adaptation
91     dTheta = -sign_b0 * e * ...
92         [ alpha1 * yf;
93           alpha2 * ydot_f;
94           alpha3 * rf ];
95
96     Theta_next = Theta + Ts * dTheta;
97
98     % State updates
99     x_ym_next = x_ym + Ts * ( A * x_ym + b * r_tilde );
100    x_yf_next = x_yf + Ts * ( A * x_yf + b * y_tilde );
101    x_rf_next = x_rf + Ts * ( A * x_rf + b * r_tilde );
102
103 end
104
105 % Memory update
106 y_prev_next = y(k);

```

Výpis kódu 7: Pseudokód diskrétnej implementácie gradientného MRAC regulátora s fázou ustálenia

5.2 Implementácia MRAC v prostredí MATLAB

Cieľom tejto časti bolo implementovať navrhnutý gradientný MRAC algoritmus v prostredí MATLAB/Simulink bez využitia štandardných blokov adaptívneho riadenia a bez preddefinovaných riadiacich štruktúr. Celý regulátor je realizovaný v rámci jedného bloku MATLAB Function, pričom regulačný zákon, referenčný model, filtračné členy aj adaptačný mechanizmus sú explicitne implementované priamo v zdrojovom kóde.

Referenčný model použitý pri implementácii regulátora má tvar

$$\frac{y_m(s)}{r(s)} = \frac{1}{s^2 + 20s + 1}. \quad (151)$$

Diskrétna implementácia regulátora je realizovaná so vzorkovacou periódou

$$T_s = 0.1 \text{ s}. \quad (152)$$

Keďže regulátor MRAC je navrhnutý pre činnosť v okolí pracovného bodu, implementácia obsahuje úvodnú fázu ustálenia. Počas tejto fázy je na vstup systému privádzaný konštantný riadiaci zásah

$$u_0 = 4, \quad (153)$$

po dobu

$$T_{\text{settle}} = 400 \text{ s}. \quad (154)$$

Po ukončení tejto fázy sú uložené hodnoty pracovného bodu výstupu y_0 a referencie r_0 . Ďalšia činnosť regulátora potom prebieha s normalizovanými veličinami definovanými vzhľadom na tento pracovný bod

$$\tilde{y}(k) = y(k) - y_0, \quad \tilde{r}(k) = r(k) - r_0, \quad \tilde{u}(k) = u(k) - u_0. \quad (155)$$

Po skončení fázy ustálenia je riadiaci zásah v každom vzorkovacom kroku vypočítaný zo vzťahu

$$u(k) = u_0 + \Theta^T(k)\tilde{\omega}(k), \quad (156)$$

kde regresný vektor má tvar

$$\tilde{\omega}(k) = \begin{bmatrix} \tilde{y}(k) \\ \dot{\tilde{y}}(k) \\ \tilde{r}(k) \end{bmatrix}. \quad (157)$$

Počas úvodnej fázy ustálenia sa adaptácia parametrov nevykonáva a stavy referenčného modelu aj filtračných blokov zostávajú nezmenené. Až po dosiahnutí pracovného bodu sa aktivuje adaptačný mechanizmus gradientného MRAC regulátora.

V implementácii je zároveň použitá saturácia riadiaceho zásahu, aby bol vstup systému obmedzený na fyzikálne realizovateľný rozsah. Hodnota riadiaceho signálu je preto limitovaná na interval $\langle 0, 10 \rangle$.

Implementácia regulátora v bloku MATLAB Function má nasledujúcu podobu.

```

1 function [u, ym, Theta_used, Theta_next, x_ym_next, x_yf_next, ...
2         x_rf_next, y_prev_next, y0_next, r0_next, sw_next] = ...
3         mrac_block(r, y, t, Theta_in, x_ym, x_yf, x_rf, y_prev, ...
4                 y0_in, r0_in, sw_in)
5
6 % Constant parameters
7 a0m = 1;
8 a1m = 20;
9 b0m = 1;
10
11 alpha1 = 14 * 0.0001;
12 alpha2 = 30 * 0.0001;
13 alpha3 = 14 * 0.0001;
```

```

14
15 sign_b0 = 1;
16 Ts = 0.1;
17
18 u0 = 4;
19 T_settle = 400;
20
21 Theta = Theta_in;
22 Theta_used = Theta;
23
24 A = [0, 1;
25      -a0m, -a1m];
26
27 b = [0;
28      1];
29
30 C_yf = [1, 0;
31         0, 1];
32
33 C_rf = [1, 0];
34
35 C_ym = [b0m, 0];
36
37 y_prev_next = y;
38 y0_next = y0_in;
39 r0_next = r0_in;
40 sw_next = sw_in;
41
42 if t < T_settle
43
44     % Settling phase
45     u = u0;
46     ym = C_ym * x_ym;
47
48     % Freeze adaptation and dynamic blocks
49     Theta_next = Theta;
50     x_ym_next = x_ym;
51     x_yf_next = x_yf;
52     x_rf_next = x_rf;
53
54 else
55
56     % Store operating point only once
57     if sw_in < 0.5
58         y0_next = y;
59         r0_next = r;
60         sw_next = 1;
61     end
62
63     % Normalized signals
64     y_tilde = y - y0_next;
65     r_tilde = r - r0_next;
66     y_prev_tilde = y_prev - y0_next;
67
68     % Numerical differentiation
69     ydot_tilde = (y_tilde - y_prev_tilde) / Ts;
70
71     % Outputs from current states
72     ym = C_ym * x_ym;
73     yf_full = C_yf * x_yf;
74     yf = yf_full(1);
75     ydot_f = yf_full(2);
76     rf = C_rf * x_rf;
77
78     % Tracking error
79     e = y_tilde - ym;
80
81     % Control law
82     omega = [y_tilde; ydot_tilde; r_tilde];
83     u_tilde = Theta.' * omega;
84     u = u0 + u_tilde;
85     u = max(min(u, 10), 0);

```

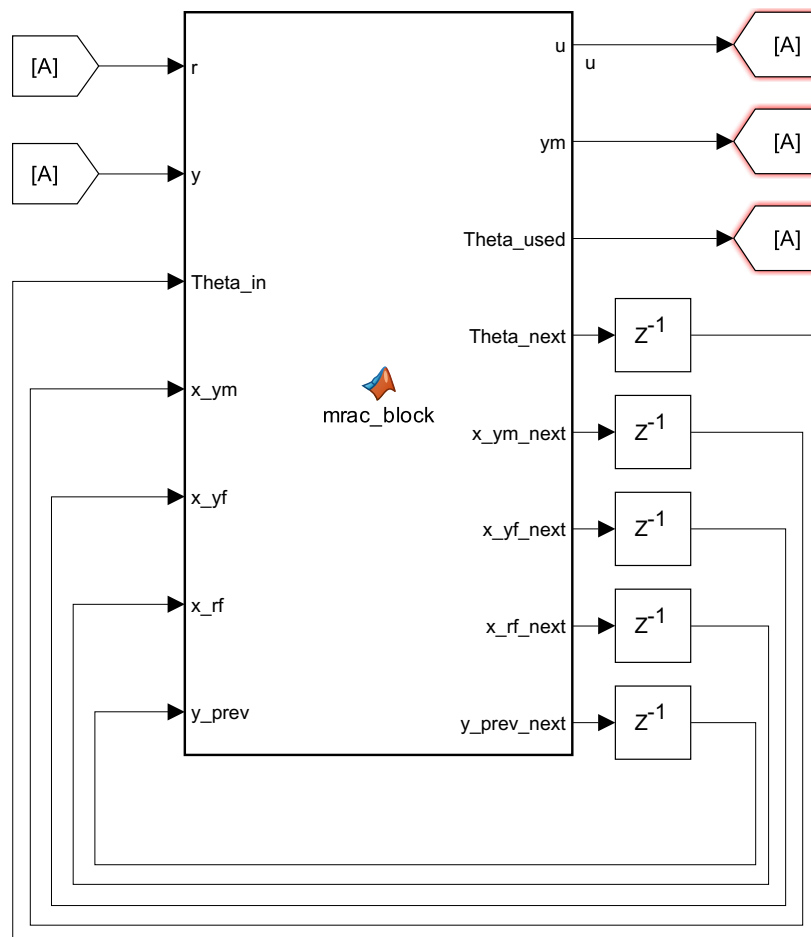
```

86
87 % MIT adaptation
88 dTheta = -sign_b0 * e * [alpha1 * yf;
89                          alpha2 * ydot_f;
90                          alpha3 * rf];
91
92 Theta_next = Theta + dTheta * Ts;
93
94 % State updates
95 x_ym_next = x_ym + Ts * (A * x_ym + b * r_tilde);
96 x_yf_next = x_yf + Ts * (A * x_yf + b * y_tilde);
97 x_rf_next = x_rf + Ts * (A * x_rf + b * r_tilde);
98
99 end
100
101 end

```

Výpis kódu 8: Implementácia gradientného MRAC regulátora v bloku MATLAB Function s fázou ustálenia

Schéma implementovaného bloku MATLAB Function v prostredí Simulink spolu s jeho vstupmi a výstupmi je znázornená na obr. 22.



Obr. 22: Blok MATLAB Function implementujúci gradientný MRAC regulátor v prostredí Simulink

5.2.1 Simulácia

V prvej fáze overovania bola realizovaná simulácia navrhnutého regulátora na systéme identifikovanom v časti 2, konkrétne na modeli (29). Cieľom simulácie bolo overiť, či

navrhnutý adaptačný mechanizmus vedie k postupnej úprave parametrov regulátora a či systém dokáže sledovať výstup referenčného modelu.

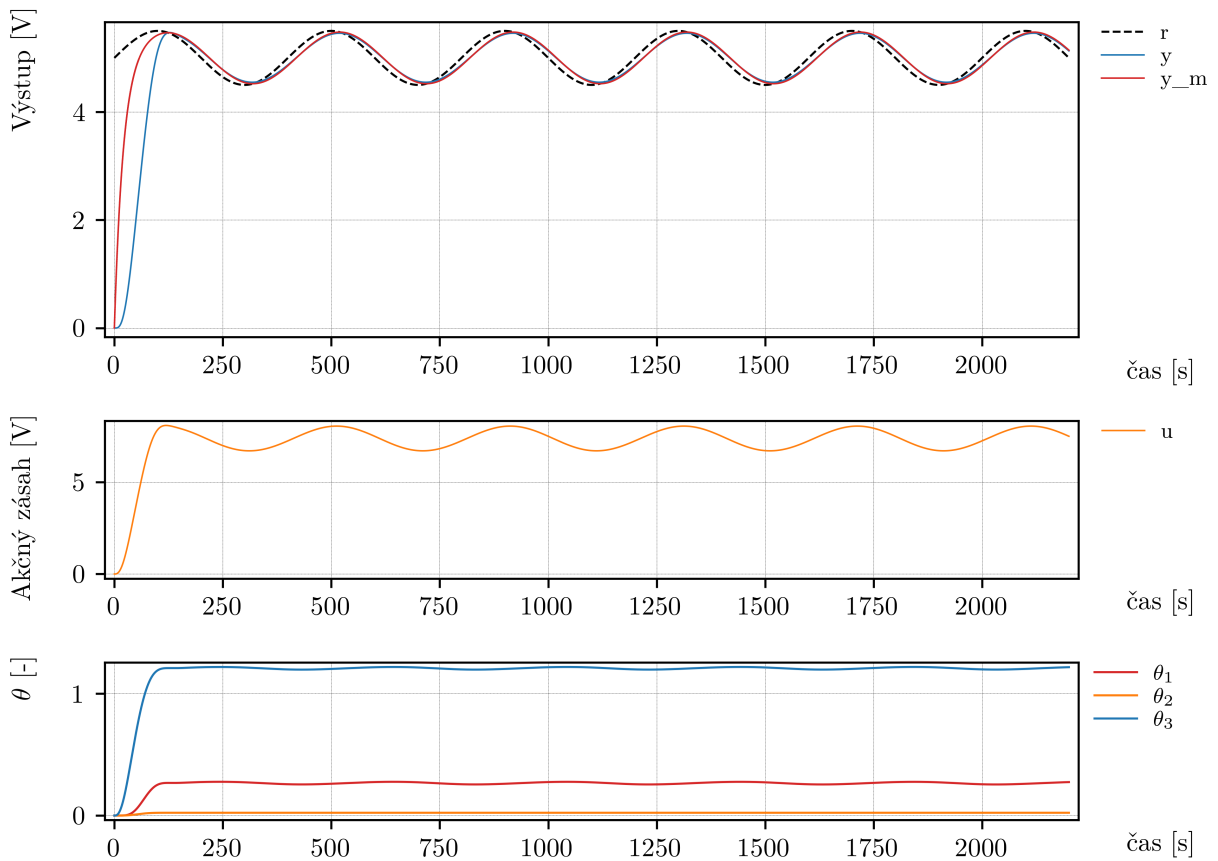
V prípade simulácie nehrala presná voľba adaptačných parametrov α_1 , α_2 , α_3 zásadnú úlohu. Dôraz bol kladený najmä na samotné overenie funkčnosti algoritmu. Počiatočné hodnoty parametrov boli zvolené nulové

$$\Theta(0) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad (158)$$

aby bolo možné priamo sledovať proces adaptácie.

Počas prvých 400 s bol systém riadený konštantným zásahom $u_0 = 4$, aby sa dostal do okolia pracovného bodu. Až po uplynutí tejto doby bola aktivovaná adaptívna časť MRAC regulátora.

Výsledok simulácie je uvedený na obr. 23. Na obrázku sú zobrazené priebehy výstupu systému $y(t)$, výstupu referenčného modelu $y_m(t)$ a referenčného signálu $r(t)$, ďalej riadiaci zásah $u(t)$ a vývoj parametrov Θ_1 , Θ_2 , Θ_3 .



Obr. 23: Simulačné výsledky gradientného MRAC regulátora

Zo získaných priebehov možno konštatovať, že po úvodnej fáze ustálenia algoritmus vykonáva adaptáciu parametrov regulátora a hodnoty parametrov sa v čase menia v súlade s minimalizáciou chyby sledovania. Výstup systému zároveň sleduje výstup referenčného modelu, čo potvrdzuje funkčnosť implementovaného gradientného MRAC algoritmu.

5.2.2 Experiment na tepelnej sústave

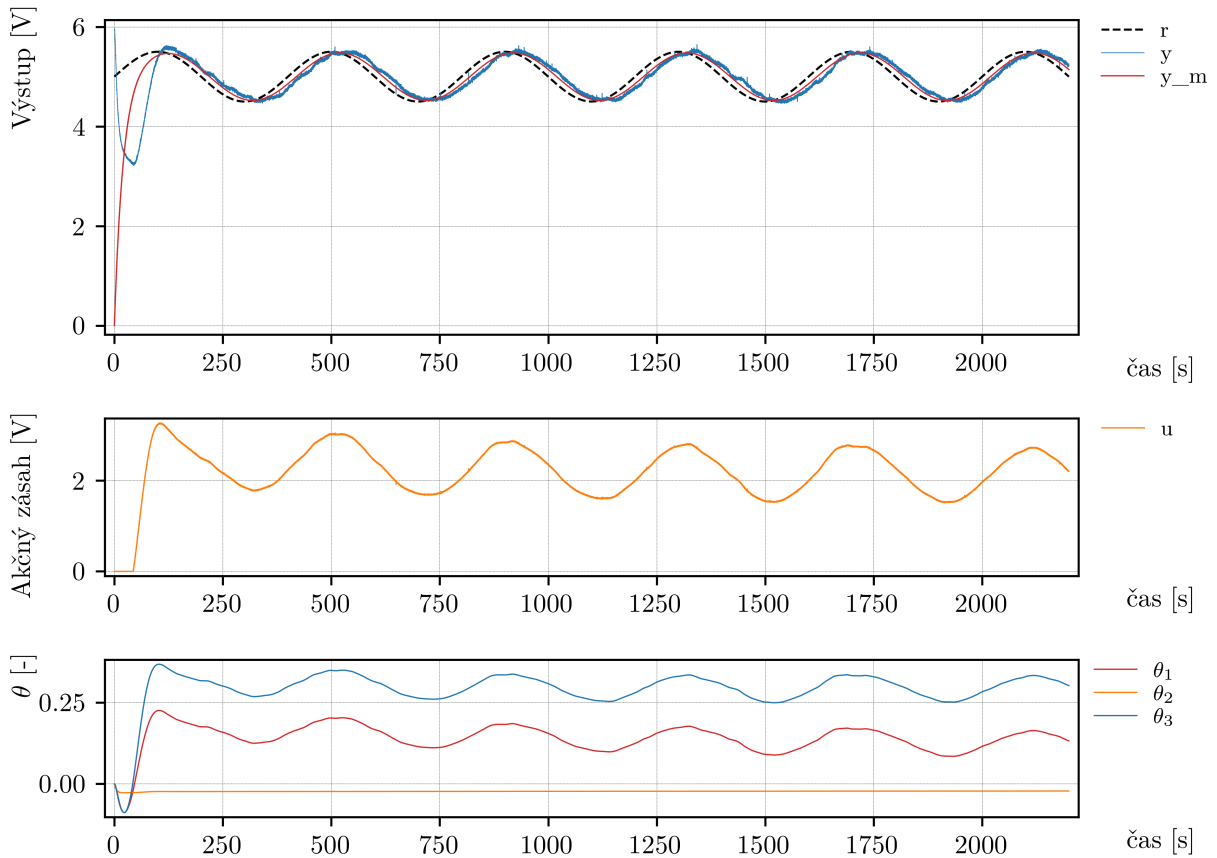
Po simulačnom overení bol regulátor aplikovaný na reálnu sústavu, konkrétne na tepelný systém.

Hodnoty adaptačných parametrov boli zvolené empiricky, teda na základe experimentálneho pozorovania správania systému. Počas ladenia boli jednotlivé koeficienty menené a následne bol vyhodnotený výsledok regulácie. Ako vhodná voľba sa ukázali hodnoty

$$[\alpha_1, \alpha_2, \alpha_3] = [14, 30, 14] \cdot 10^{-4}.$$

Aj v tomto prípade boli počiatočné hodnoty parametrov zvolené nulové.

Výsledok experimentu je uvedený na obr. 24. Na obrázku sú opäť zobrazené priebehy výstupu systému, referenčného modelu a referenčného signálu, ďalej riadiaci zásah a vývoj parametrov regulátora.



Obr. 24: Experimentálne výsledky gradientného MRAC regulátora na tepelnej sústave

Z experimentálnych výsledkov možno pozorovať, že adaptácia parametrov Θ prebieha očakávaným spôsobom. Výstup systému sleduje výstup referenčného modelu s uspokojivou presnosťou, pričom je možné pozorovať mierne zaostávanie za referenčným priebehom. Tento jav je spôsobený pomalšou dynamikou reálnej tepelnej sústavy v porovnaní s ideálnym modelom použitým pri simulácii.

Literatúra

- [1] M. Fikar, J. Mikleš, *Identifikácia systémov*. Slovenská technická univerzita v Bratislave, 1998. ISBN 80-227-1177-2.
- [2] M. Tárník, Učebný text KUT015: *Laboratórne zariadenie TS: orientačný prehľad*. Slovenská technická univerzita v Bratislave, 2025. Dostupné online: <https://github.com/OkoliePracovnehoBodu/KUT/>
- [3] D. C. Montgomery, E. A. Peck, G. G. Vining, *Introduction to Linear Regression Analysis, Fifth Edition*. John Wiley & Sons, Inc., Hoboken, New Jersey, 2012. ISBN 978-0-470-54281-1.